SIMATS ENGINEERING

ASSESSMENT TOOL 1 - HIGH

(ANALYTICAL PROBLEM SOLVING)

COURSE CODE: ITA0402

COURSE NAME: STATISTICS WITH R PROGRAMMING

COURSE OUTCOME COVERED

CO3: *Construct and manipulate data frames using reshaping, merging, and file I/O operations in R to prepare structured datasets for analysis. (BL3, BL5)*

Assessment Tool Weightage: 20%

QUESTIONS: 10 **Total Marks: 100**

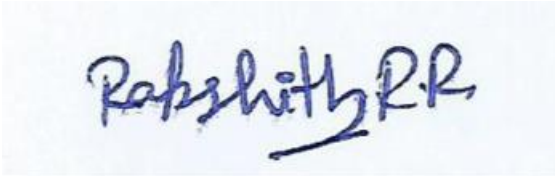
ANNEXURE A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

*I **RAKSHITH R.R (192324156)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



1. Making Data Frames

You receive raw sensor log vectors (sensor_id as character, voltage as numeric, status as logical) of unequal lengths ($N_1 = 120$), ($N_2 = 119$), ($N_3 = 120$)) from an embedded diagnostic stream. If you pass these directly into the data.frame() constructor, R triggers a recycling-rule exception or silent mismatch warning. Analyze the structural failure mechanism of R's internal memory allocation during this coercion, and write an R routine that programmatically inspects and rectifies vector asymmetry by padding the shortest vector with NA *only after* generating a diagnostic discrepancy report detailing the missing index positions.

2. Working with Data Frames

An engineering telemetry frame contains a continuous numeric column Temperature with intermittent anomalies coded as literal character strings "ERR_99" instead of standard numeric NA values. Differentiate the impact this has on vectorized operations like mean(df$Temperature, na.rm = TRUE) (which fails due to character coercion) versus indexing subsets. Construct an analytical data-wrangling script that isolates non-numeric string tokens, logs their row indices and exact token frequency, replaces them with explicit NA, and safely type-casts the column back to double precision without truncating valid numeric records.

3. Data Reshaping

A structural strain matrix records measurements across 4 spatial coordinate sensors over 3 time steps, but the data is spread across multi-indexed columns (SensorA_X, SensorA_Y, etc.). Deconstruct why maintaining this wide format creates a dimensional restriction for spatial regression models requiring independent observation rows. Use base R or reshape mechanics to transpose and restructure the coordinate attributes into a cohesive time-series layout without losing row-to-observation mapping integrity.

4. Melting and Casting of Data

Using the reshape2 package, you apply melt() on a multi-sensor performance matrix. However, duplicate ID combinations exist for identical timestamps due to dual-bitrate

sampling redundancy. If you execute dcast() directly on this molten data frame without an aggregation function, R throws a structural ambiguity error. Analyze why multiple data points mapping to a single cell require an explicit reduction function (fun.aggregate), and write a dcast command that computes both the mean and variance simultaneously to preserve variance analysis during the wide-format conversion.

5. Merging Data Frames

You are tasked with fusing df_control (keyed by SystemID and Timestamp) and df_telemetry (keyed by SystemID and Timestamp). Both tables contain a column named Status, which results in conflicting .x and .y suffixes upon a standard inner join via merge(). Furthermore, time stamps in the two tables drift by up to 2 seconds due to asynchronous clock skews, causing an inner join on exact values to drop 40% of records. Analyze how this data loss biases the system reliability estimation, and design a custom interval-matching strategy or pre-processing rounding mechanism before executing an outer merge that preserves unmatched orphan error logs.

Data Frames

Q1. Analyze a dataset of 50,000 student records stored in a data frame. Identify the most efficient method to detect duplicate entries and justify your approach.
Q2. A data frame contains missing values across multiple columns. Evaluate different strategies for handling missing data and recommend the best solution.
Q3. Given a sales data frame with inconsistent data types, analyze the impact on statistical calculations and propose corrective actions.
Q4. Compare two large data frames containing customer transactions. Determine how to identify data integrity issues and evaluate their causes.
Q5. A company reports conflicting results from the same data frame. Analyze possible reasons and design a validation framework.

Making Data Frames

Q1. Design a data frame structure for a smart-city traffic monitoring system and justify the selection of attributes.
Q2. Evaluate two alternative data frame designs for storing IoT sensor data and recommend the most scalable model.

Q3. Analyze the consequences of assigning incorrect data types while creating a data frame for healthcare records.
Q4. Create a strategy for constructing a data frame from multiple heterogeneous data sources while preserving consistency.
Q5. Assess the efficiency of creating a data frame from large vectors versus importing directly from files.

Working with Data Frames

Q1. Analyze a scenario where filtering operations on a large data frame are producing unexpected results. Determine the root cause.
Q2. Evaluate different methods for sorting a million-row data frame and justify the most efficient approach.
Q3. A data analyst performs multiple transformations on a data frame. Analyze how errors can propagate through the workflow.
Q4. Compare indexing and logical subsetting techniques for extracting data and evaluate their performance trade-offs.
Q5. Design a workflow to monitor and validate modifications made to a shared data frame by multiple users.

Data Reshaping

Q1. Analyze a business dataset before and after reshaping. Evaluate how the transformation affects data interpretation.
Q2. A dataset must be reshaped for machine learning. Determine which structure is most suitable and justify your decision.
Q3. Evaluate the risks of reshaping data incorrectly in a healthcare analytics application.
Q4. Compare wide and long data formats for time-series analysis and recommend the better approach for forecasting.
Q5. Design a reshaping strategy for integrating survey responses collected across multiple years.

Melting and Casting of Data

Q1. Analyze a dataset where repeated measurements are stored in multiple columns. Determine how melting improves analysis.
Q2. Evaluate the impact of casting data into different summary structures for business reporting.

Q3. Compare two casting approaches that produce different aggregation results and explain the reasons.

Q4. Design a workflow using melting and casting to prepare data for predictive modeling.

Q5. Assess the challenges of applying melt and cast operations on large-scale industrial datasets.

Merging Data Frames

Q1. Analyze the consequences of performing an incorrect join operation between two financial datasets.

Q2. Evaluate inner, outer, left, and right joins for integrating customer and transaction data.

Q3. A merged data frame contains unexpected duplicate rows. Investigate the possible causes and propose solutions.

Q4. Design a strategy to merge data frames from multiple departments while ensuring data consistency.

Q5. Compare key-based merging and index-based merging techniques for large engineering datasets.

RUBRICS FOR CALCULATION

Class Test					
Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Understanding of Data Frame Concepts	25	Demonstrates thorough understanding of Data Frame concepts and operations.	Demonstrates good understanding with minor omissions.	Demonstrates basic understanding with limited details.	Demonstrates poor or incorrect understanding of Data Frame concepts.
Understanding of Reshaping and Merging Techniques	25	Clearly explains reshaping and merging concepts with appropriate use cases.	Explains concepts correctly with minor gaps.	Provides partial understanding of reshaping or merging concepts.	Unable to explain reshaping or merging concepts correctly.
Knowledge of File I/O Operations	20	Accurately explains file handling functions and their usage.	Explains most file handling concepts correctly.	Provides limited explanation of file operations.	Demonstrates little understanding of file I/O operations.
Application to Data Analysis and Dataset Preparation	20	Effectively applies concepts to data preparation and analysis tasks.	Provides appropriate applications with adequate explanation.	Gives limited applications or examples.	Unable to relate concepts to practical applications.
Technical Accuracy and Completeness	10	Uses correct terminology and provides complete answers.	Mostly accurate with minor technical errors.	Some inaccuracies and incomplete responses.	Major technical errors and incomplete answers.

Part A: Scenario-Based Questions (Questions 1 to 5)

Each question below contains the analysis of the problem, the R program, the program output (as text), the output screenshots exactly as obtained from RGui, and the conclusion. The task describes the situations but does not supply data files, so every program creates its own small data set in R (using set.seed() where random values are used) so that the results can be reproduced exactly each time the code is run.

Question 1: Making Data Frames

You receive raw sensor log vectors (sensor_id as character, voltage as numeric, status as logical) of unequal lengths (\(N_1 = 120\), \(N_2 = 119\), \(N_3 = 120\)) from an embedded diagnostic stream. If you pass these directly into the data.frame() constructor, R triggers a recycling-rule exception or silent mismatch warning. Analyze the structural failure mechanism of R's internal memory allocation during this coercion, and write an R routine that programmatically inspects and rectifies vector asymmetry by padding the shortest vector with NA only after generating a diagnostic discrepancy report detailing the missing index positions.

Analysis:

Why the direct call fails. A data frame is a list of atomic vectors that must all have the same number of rows. data.frame() first finds the longest column (120) and can recycle a shorter column only when the longest length is an exact multiple of the shorter one. Here 120 %% 119 = 1, so R refuses to build the object and stops with the error "arguments imply differing number of rows: 120, 119"; no partial data frame is created. Each column is stored as one contiguous block of fixed length in memory, so R never stretches a short vector by itself.

The dangerous case. If the lengths had divided evenly (for example 60 and 120), R would recycle the short vector silently and repeat its values, producing a data frame that looks valid but contains wrong readings. This is why the lengths must be inspected before the data frame is built.

Routine designed. diagnose_and_pad() (1) measures the length of every vector, (2) prints a discrepancy report with the class, length, number missing and the exact missing index positions, (3) tests whether recycling would be possible, and only then (4) pads every shorter vector to the target length with length(v) <- max_len, which adds NA of the correct type (NA_real_ for numeric, NA_character_ for character, NA for logical) so that column classes are not changed.

Caution. Padding at the end is valid only when the missing reading really is the last one. If a value is missing in the middle of the stream, the sensor_id or timestamp should be used to align the rows first; otherwise later rows would be shifted.

Code:

```
# Q1: Making Data Frames - unequal sensor vectors (diagnose, then pad with NA)
set.seed(101)
sensor_id <- paste0("S", sprintf("%03d", 1:120))                    # N1 = 120 (character)
```

```
voltage   <- round(rnorm(119, mean = 3.3, sd = 0.15), 3)                # N2 = 119 (numeric)
status    <- sample(c(TRUE, FALSE), 120, replace = TRUE, prob = c(0.9, 0.1)) # N3 = 120 (logical)


# Step 1: direct construction fails
cat("Direct data.frame() call result:\n")
direct <- tryCatch(data.frame(sensor_id, voltage, status),
                   error = function(e) conditionMessage(e))
print(direct)


# Step 2: routine that reports the discrepancy and then pads with NA
diagnose_and_pad <- function(cols) {
 lens    <- sapply(cols, length)
 max_len <- max(lens)
 cat("\n---- DIAGNOSTIC DISCREPANCY REPORT ----\n")
 report <- data.frame(Vector = names(cols), Class = sapply(cols, class),
                      Length = lens, Missing = max_len - lens, row.names = NULL)
 print(report)
 cat("Target length (longest vector):", max_len, "\n")
 for (nm in names(cols)) {
  if (lens[nm] < max_len) {
   miss <- (lens[nm] + 1):max_len
   cat("Vector '", nm, "' is short by ", length(miss),
       " element(s); missing index position(s): ", paste(miss, collapse = ", "), "\n", sep = "")
   cat("Can R recycle it automatically? ", max_len %% lens[nm] == 0, "\n")
  }
 }
 padded <- lapply(cols, function(v) { length(v) <- max_len; v })  # pads with NA, keeps type
 data.frame(padded, stringsAsFactors = FALSE)
}


df <- diagnose_and_pad(list(sensor_id = sensor_id, voltage = voltage, status = status))


cat("\nRectified data frame dimensions:", dim(df), "\n")
cat("\nStructure:\n"); str(df)
cat("\nNA count per column:\n"); print(colSums(is.na(df)))
cat("\nLast 3 rows (padded NA visible):\n"); print(tail(df, 3))
```

Output:

```
Direct data.frame() call result:
[1] "arguments imply differing number of rows: 120, 119"


---- DIAGNOSTIC DISCREPANCY REPORT ----
    Vector     Class Length Missing
1 sensor_id character    120       0
2   voltage   numeric    119       1
3    status   logical    120       0
Target length (longest vector): 120
Vector 'voltage' is short by 1 element(s); missing index position(s): 120
Can R recycle it automatically?  FALSE

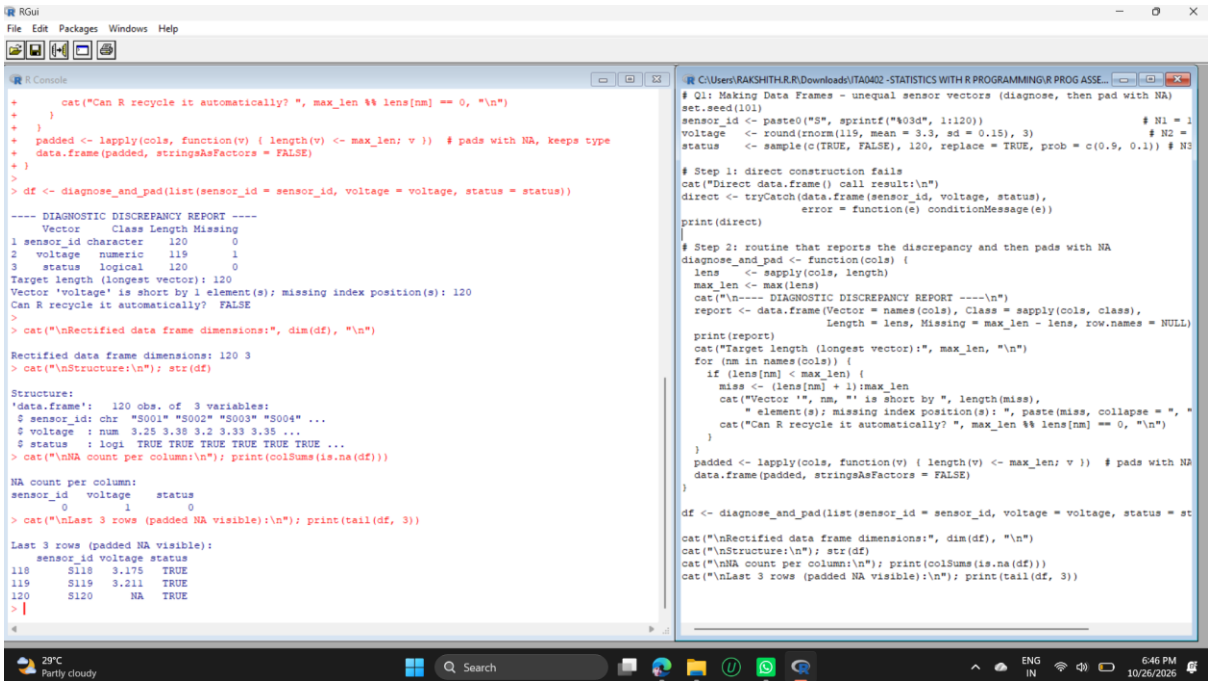

Rectified data frame dimensions: 120 3


Structure:
'data.frame':   120 obs. of  3 variables:
 $ sensor_id: chr  "S001" "S002" "S003" "S004" ...
 $ voltage  : num  3.25 3.38 3.2 3.33 3.35 ...
 $ status   : logi  TRUE TRUE TRUE TRUE TRUE TRUE ...


NA count per column:
sensor_id   voltage    status
        0         1         0


Last 3 rows (padded NA visible):
   sensor_id voltage status
118      S118   3.175   TRUE
119      S119   3.211   TRUE
120      S120      NA   TRUE
```

Output Screenshot:



Conclusion:

The report shows that voltage is one element short (length 119 against 120) and that index position 120 is missing. After padding, the data frame has 120 rows x 3 columns, the classes are preserved (character, numeric, logical) and the only NA is voltage in row 120, so no valid record was changed. The rectified frame can be saved for later steps with write.csv(df, "sensor_log_clean.csv", row.names = FALSE) and re-imported with read.csv(..., colClasses = c("character", "numeric", "logical")) so the types stay correct.

Question 2: Working with Data Frames

An engineering telemetry frame contains a continuous numeric column Temperature with intermittent anomalies coded as literal character strings "ERR_99" instead of standard numeric NA values. Differentiate the impact this has on vectorized operations like mean(df$Temperature, na.rm = TRUE) (which fails due to character coercion) versus indexing subsets. Construct an analytical data-wrangling script that isolates non-numeric string tokens, logs their row indices and exact token frequency, replaces them with explicit NA, and safely type-casts the column back to double precision without truncating valid numeric records.

Analysis:

Effect of the string tokens. R columns hold one type only. A single text value such as "ERR_99" forces the whole Temperature column to character (the coercion order is logical, integer, double, character). Therefore mean(df$Temperature, na.rm = TRUE) does not work: na.rm removes only real NA values, and the text tokens are not NA, so R gives a warning ("argument is not numeric or logical: returning NA") and returns NA. Arithmetic such as df$Temperature + 1 fails with "non-numeric argument to binary operator".

Effect on indexing and subsets. Positional indexing such as df[4,] still works because it does not depend on the type. Logical subsetting, however, does not fail - it silently gives wrong answers: df$Temperature > 100 compares text alphabetically, so even "93.3" > "100" is TRUE and all 30 rows are returned. Sorting is also alphabetical. Silent wrong results are more dangerous than an error.

Wrangling script. (1) as.numeric() is tried inside suppressWarnings() to find the entries that cannot be converted; (2) tokens that become NA although they were not NA before are the non-numeric strings, so which() gives their row indices and table() gives the exact token frequency; (3) they are replaced with explicit NA; (4) the column is converted with as.numeric(), which stores double precision values and therefore does not truncate valid readings; (5) checks confirm the result.

Code:

```
# Q2: Working with Data Frames - clean "ERR_99" tokens in Temperature
set.seed(7)
temp <- round(rnorm(30, mean = 75, sd = 8), 2)
temp_chr <- as.character(temp)
temp_chr[c(4, 11, 12, 25, 29)] <- "ERR_99"
df <- data.frame(Reading = 1:30, Temperature = temp_chr, stringsAsFactors = FALSE)
cat("Column class before cleaning:", class(df$Temperature), "\n")
print(head(df, 6))


# --- Impact of the string tokens ---
cat("\n[1] mean() on the character column:\n")
m <- withCallingHandlers(mean(df$Temperature, na.rm = TRUE),
```

```
    warning = function(w) { cat("  Warning:", conditionMessage(w), "\n");
invokeRestart("muffleWarning") })
cat("   Result:", m, "\n")


cat("\n[2] Subsetting Temperature > 100 on the character column:\n")
cat("   Rows returned:", nrow(df[df$Temperature > 100, ]),
   "(text is compared alphabetically, so '93.3' > '100' is TRUE)\n")


# --- Wrangling script ---
num_try  <- suppressWarnings(as.numeric(df$Temperature))
bad_idx  <- which(is.na(num_try) & !is.na(df$Temperature))   # non-numeric tokens
tokens   <- df$Temperature[bad_idx]


cat("\nRow indices of non-numeric tokens:", bad_idx, "\n")
cat("Token frequency:\n"); print(table(tokens))


df$Temperature[bad_idx] <- NA                    # explicit NA
df$Temperature <- as.numeric(df$Temperature)       # safe type-cast to double


cat("\nColumn class after cleaning:", class(df$Temperature), "\n")
cat("Valid numeric records kept :", sum(!is.na(df$Temperature)), "of", nrow(df), "\n")
cat("Records replaced with NA   :", sum(is.na(df$Temperature)), "\n")
cat("Original values preserved  :", all.equal(df$Temperature[-bad_idx], temp[-bad_idx]), "\n")
cat("Mean Temperature (na.rm=TRUE):", round(mean(df$Temperature, na.rm = TRUE), 3), "\n")
cat("Rows with Temperature > 85  :", nrow(df[which(df$Temperature > 85), ]), "\n")
str(df)
```

Output:

```
Column class before cleaning: character
  Reading Temperature
1       1        93.3
2       2       65.43
3       3       69.45
4       4      ERR_99
5       5       67.23
6       6       67.42
```

[1] mean() on the character column:

Warning: argument is not numeric or logical: returning NA

Result: NA

[2] Subsetting Temperature > 100 on the character column:

Rows returned: 30 (text is compared alphabetically, so '93.3' > '100' is TRUE)

Row indices of non-numeric tokens: 4 11 12 25 29

Token frequency:

tokens

ERR_99

5

Column class after cleaning: numeric

Valid numeric records kept : 25 of 30

Records replaced with NA : 5

Original values preserved : TRUE

Mean Temperature (na.rm=TRUE): 77.854

Rows with Temperature > 85 : 5

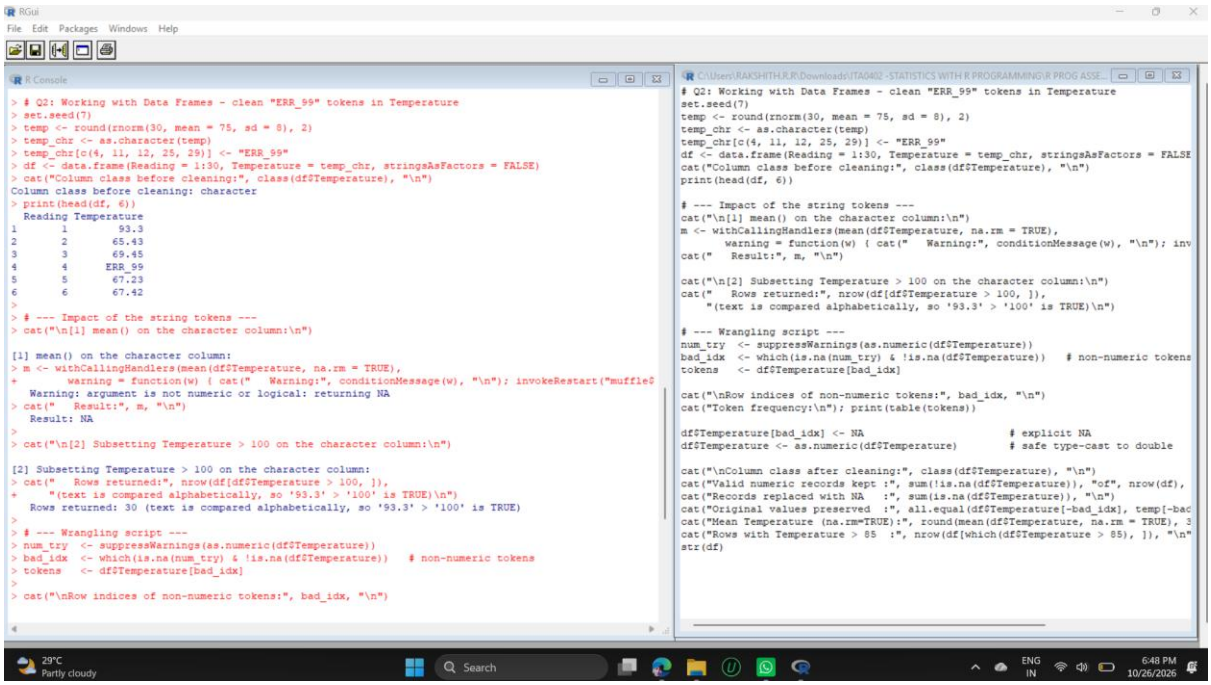
'data.frame': 30 obs. of 2 variables:

$ Reading : int 1 2 3 4 5 6 7 8 9 10 ...

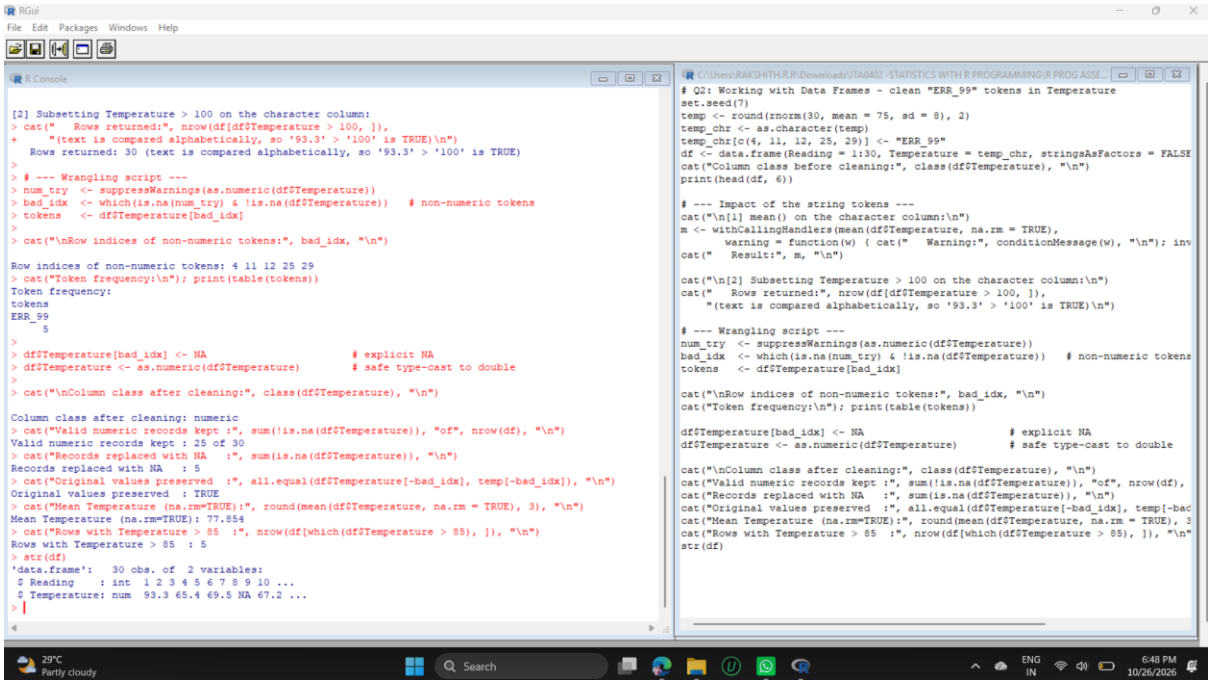
$ Temperature: num 93.3 65.4 69.5 NA 67.2 ...

Output Screenshots:

Screenshot 1 of 2



Screenshot 2 of 2



Conclusion:

Five "ERR_99" tokens were found in rows 4, 11, 12, 25 and 29 (frequency table: ERR_99 = 5). After cleaning, the column is numeric, 25 of 30 records are valid, 5 are NA, and all.equal() confirms that the 25 valid values are exactly the original numbers. The mean temperature is now computed correctly as 77.854, and a filter such as Temperature > 85 returns the correct 5 rows.

Question 3: Data Reshaping

A structural strain matrix records measurements across 4 spatial coordinate sensors over 3 time steps, but the data is spread across multi-indexed columns (SensorA_X, SensorA_Y, etc.). Deconstruct why maintaining this wide format creates a dimensional restriction for spatial regression models requiring independent observation rows. Use base R or reshape mechanics to transpose and restructure the coordinate attributes into a cohesive time-series layout without losing row-to-observation mapping integrity.

Analysis:

Why the wide format is restrictive. In the wide layout there is one row per time step (3 rows) and 12 separate columns (SensorA_X, SensorA_Y, SensorA_Strain, SensorB_X, ...). One row therefore contains four different sensor observations, and the same variable (X, Y or Strain) is spread over four columns whose sensor identity is hidden in the column name. A spatial regression needs one response column and one set of predictor columns with one independent observation per row. With the wide layout, the model cannot refer to "X" or "Strain" generically, the number of observations appears as 3 instead of 12, and adding a sensor changes the number of columns.

Restructuring. stats::reshape() with direction = "long" is used with varying = list(X columns, Y columns, Strain columns), v.names = c("X", "Y", "Strain"), timevar = "Sensor", times = A to D and idvar = "Time_Step". Because the three varying lists are given in the same sensor order, each X, Y and Strain value stays together with its own Time_Step and Sensor. The result is ordered by sensor and time to give a clean time-series layout per sensor.

Integrity checks. The script checks that the long table has 4 x 3 = 12 rows and then compares every X, Y and Strain value in the long table with its original cell in the wide table.

Code:

```
# Q3: Data Reshaping - wide strain matrix to long (time-series) layout
set.seed(42)
sensors <- c("A", "B", "C", "D")
wide <- data.frame(Time_Step = 1:3)
for (s in sensors) {
  k <- match(s, sensors)
  wide[[paste0("Sensor", s, "_X")]]      <- round(10 * k + rnorm(3, 0, 0.2), 2)
  wide[[paste0("Sensor", s, "_Y")]]      <- round(5 * k + rnorm(3, 0, 0.2), 2)
  wide[[paste0("Sensor", s, "_Strain")]] <- round(100 + 20 * k + rnorm(3, 0, 3), 1)
}
cat("WIDE format (", nrow(wide), "rows x", ncol(wide), "columns):\n"); print(wide)


long <- reshape(wide, direction = "long",
                varying = list(paste0("Sensor", sensors, "_X"),
```

```
                    paste0("Sensor", sensors, "_Y"),
                    paste0("Sensor", sensors, "_Strain")),
          v.names = c("X", "Y", "Strain"),
          timevar = "Sensor", times = sensors,
          idvar = "Time_Step")
long <- long[order(long$Sensor, long$Time_Step), ]
rownames(long) <- NULL
cat("\nLONG format (", nrow(long), "rows x", ncol(long), "columns):\n"); print(long)


# --- Integrity checks: every wide value must appear in the matching long row ---
ok <- TRUE
for (s in sensors) for (t in 1:3) {
  row_long <- long[long$Sensor == s & long$Time_Step == t, ]
  ok <- ok && row_long$X      == wide[wide$Time_Step == t, paste0("Sensor", s, "_X")] &&
          row_long$Y      == wide[wide$Time_Step == t, paste0("Sensor", s, "_Y")] &&
          row_long$Strain == wide[wide$Time_Step == t, paste0("Sensor", s, "_Strain")]
}
cat("\nRow count check (4 sensors x 3 time steps = 12):", nrow(long) == 12, "\n")
cat("Value-by-value mapping preserved           :", ok, "\n")


# --- Long format feeds a regression directly (one observation per row) ---
model <- lm(Strain ~ X + Y, data = long)
cat("\nSpatial regression on the long format (Strain ~ X + Y):\n")
print(round(coef(summary(model)), 3))
```

Output:

WIDE format (3 rows x 13 columns):

Time_Step	SensorA_X	SensorA_Y	SensorA_Strain	SensorB_X	SensorB_Y	SensorB_Strain	SensorC_X	SensorC_Y
1	10.27	5.13	124.5	19.99	9.72	141.9	29.51	14.64
2	9.89	5.08	119.7	20.26	9.94	139.1	30.26	14.97
3	10.07	4.98	126.1	20.46	9.97	132.0	29.94	15.24

SensorC_Strain	SensorD_X	SensorD_Y	SensorD_Strain
165.7	39.65	20.09	178.2
158.7	40.09	20.14	181.5
159.2	39.87	20.21	174.8

LONG format (12 rows x 5 columns):

	Time_Step	Sensor	X	Y	Strain
1	1	A	10.27	5.13	124.5
2	2	A	9.89	5.08	119.7
3	3	A	10.07	4.98	126.1
4	1	B	19.99	9.72	141.9
5	2	B	20.26	9.94	139.1
6	3	B	20.46	9.97	132.0
7	1	C	29.51	14.64	165.7
8	2	C	30.26	14.97	158.7
9	3	C	29.94	15.24	159.2
10	1	D	39.65	20.09	178.2
11	2	D	40.09	20.14	181.5
12	3	D	39.87	20.21	174.8

Row count check (4 sensors x 3 time steps = 12): TRUE

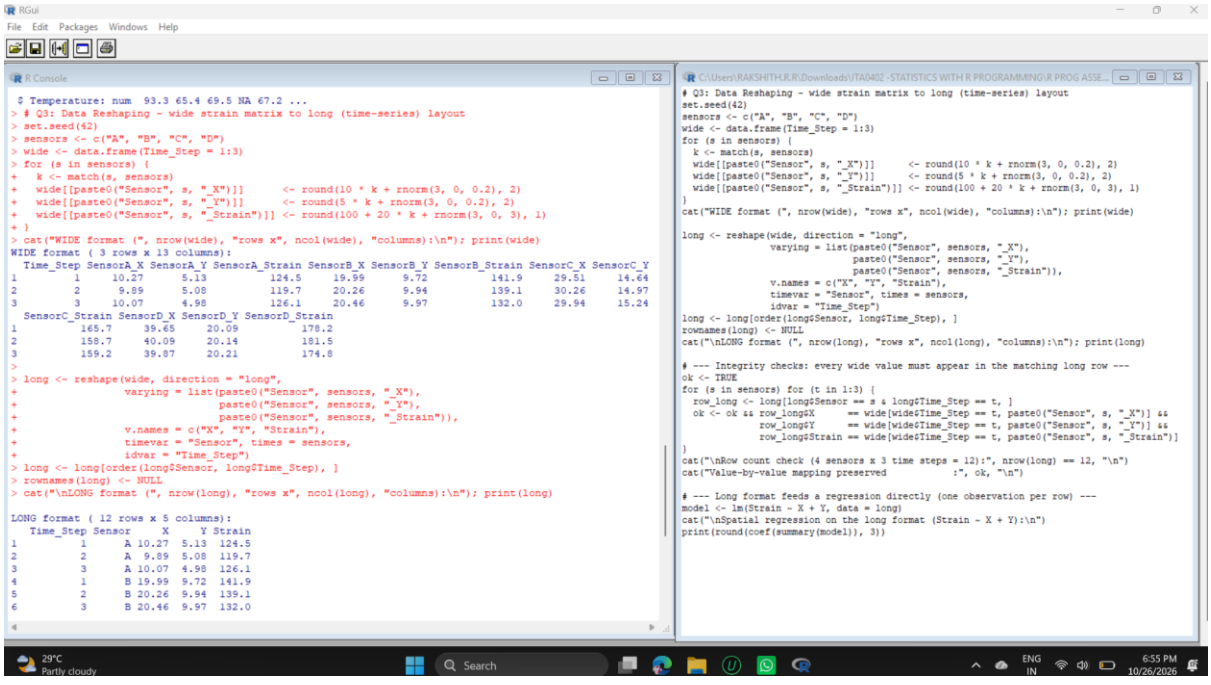
Value-by-value mapping preserved : TRUE

Spatial regression on the long format (Strain ~ X + Y):

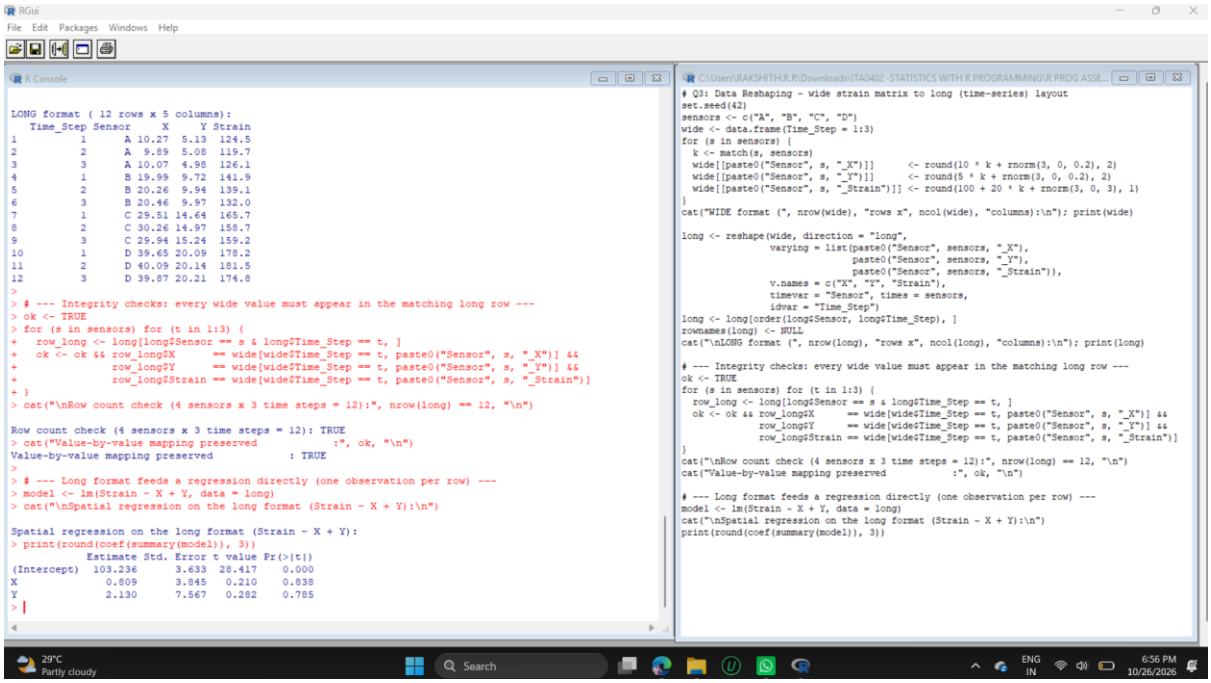
	Estimate	Std. Error	t value	Pr(>|t|)
(Intercept)	103.236	3.633	28.417	0.000
X	0.809	3.845	0.210	0.838
Y	2.130	7.567	0.282	0.785

Output Screenshots:

Screenshot 1 of 2



Screenshot 2 of 2



Conclusion:

The wide table (3 rows x 13 columns) became a long table (12 rows x 5 columns: Time_Step, Sensor, X, Y, Strain). Both checks returned TRUE (row count = 12 and value-by-value mapping preserved), so no row-to-observation mapping was lost. The long layout can be used directly in lm(Strain ~ X + Y). The regression output here is only a demonstration of the structure; with 12 simulated points the coefficients are not significant, and in a real study the repeated readings of the same sensor would need a model that allows for that dependence.

Question 4: Melting and Casting of Data

Using the reshape2 package, you apply melt() on a multi-sensor performance matrix. However, duplicate ID combinations exist for identical timestamps due to dual-bitrate sampling redundancy. If you execute dcast() directly on this molten data frame without an aggregation function, R throws a structural ambiguity error. Analyze why multiple data points mapping to a single cell require an explicit reduction function (fun.aggregate), and write a dcast command that computes both the mean and variance simultaneously to preserve variance analysis during the wide-format conversion.

Analysis:

Why an aggregation function is needed. After melt(), each row is one reading identified by Timestamp, Bitrate and Sensor. dcast(Timestamp ~ Sensor) tries to put one value in each Timestamp x Sensor cell. Because of the dual-bitrate sampling, each cell receives two readings (the frequency table shows 2 for every combination), so the cell is ambiguous: R must be told how to reduce several values to one.

What reshape2 really does. In the output, dcast() did not stop with an error. It printed the message "Aggregation function missing: defaulting to length" and returned the number of readings (2) in every cell instead of the readings themselves. This is more dangerous than an error, because the table looks valid but the measurements have been replaced by counts. An explicit fun.aggregate is therefore mandatory whenever the ID combination is not unique.

Mean and variance together. fun.aggregate in dcast() must return a single value per cell, so two summaries cannot be returned from one function. The solution is to cast once with mean and once with var, rename the columns (_mean, _var) and merge by Timestamp. A second version adds a Statistic column (mean/var) to the summarised data and uses a single dcast(Timestamp ~ Sensor + Statistic) call, giving the same wide table.

Code:

```
# Q4: Melting and Casting with reshape2 (duplicate IDs -> need fun.aggregate)
if (!requireNamespace("reshape2", quietly = TRUE)) install.packages("reshape2")  # one-time install
library(reshape2)
set.seed(11)
perf <- data.frame(Timestamp = rep(1:4, each = 2),          # 2 readings per timestamp
                   Bitrate   = rep(c("Low", "High"), times = 4),
                   Sensor1   = round(rnorm(8, 50, 5), 1),
                   Sensor2   = round(rnorm(8, 60, 6), 1),
                   Sensor3   = round(rnorm(8, 70, 7), 1))
cat("Original performance matrix (duplicate Timestamps):\n"); print(perf)


molten <- melt(perf, id.vars = c("Timestamp", "Bitrate"),
               variable.name = "Sensor", value.name = "Reading")
```

```
cat("\nMolten data (first 6 of", nrow(molten), "rows):\n"); print(head(molten, 6))


cat("\nTimestamp x Sensor combinations that occur more than once:\n")
print(table(molten$Timestamp, molten$Sensor))


# dcast WITHOUT fun.aggregate
cat("\ndcast() without fun.aggregate:\n")
no_agg <- withCallingHandlers(dcast(molten, Timestamp ~ Sensor, value.var = "Reading"),
        message = function(m) { cat("  Message:", conditionMessage(m)); invokeRestart("muffleMessage") },
        warning = function(w) { cat("  Warning:", conditionMessage(w), "\n"); invokeRestart("muffleWarning") })
print(no_agg)


# dcast with explicit aggregation: mean and variance
mean_wide <- dcast(molten, Timestamp ~ Sensor, value.var = "Reading", fun.aggregate = mean)
var_wide  <- dcast(molten, Timestamp ~ Sensor, value.var = "Reading", fun.aggregate = var)
names(mean_wide)[-1] <- paste0(names(mean_wide)[-1], "_mean")
names(var_wide)[-1]  <- paste0(names(var_wide)[-1],  "_var")
combined <- merge(mean_wide, var_wide, by = "Timestamp")
cat("\nWide format with mean AND variance preserved:\n"); print(round(combined, 3))


# single-call version: add a Statistic dimension before casting
stats <- do.call(rbind, lapply(c(mean = mean, var = var), function(f)
  aggregate(Reading ~ Timestamp + Sensor, data = molten, FUN = f)))
stats$Statistic <- rep(c("mean", "var"), each = nrow(stats) / 2)
cat("\nSingle dcast over Sensor + Statistic:\n")
print(round(dcast(stats, Timestamp ~ Sensor + Statistic, value.var = "Reading"), 3))
```

Output:

```
Installing package into ‘C:/Users/RAKSHITH.R.R/AppData/Local/R/win-library/4.6’
(as ‘lib’ is unspecified)
--- Please select a CRAN mirror for use in this session ---
also installing the dependency ‘plyr’

trying URL 'https://cran.isid.ac.in/bin/windows/contrib/4.6/plyr_1.8.9.zip'
trying URL 'https://cran.isid.ac.in/bin/windows/contrib/4.6/reshape2_1.4.5.zip'
package ‘plyr’ successfully unpacked and MD5 sums checked
```

package ‘reshape2’ successfully unpacked and MD5 sums checked

The downloaded binary packages are in

C:\Windows\Temp\Rtmpk9rruj\downloaded_packages

Original performance matrix (duplicate Timestamps):

	Timestamp	Bitrate	Sensor1	Sensor2	Sensor3
1	1	Low	47.0	59.7	68.4
2	1	High	50.1	54.0	76.2
3	2	Low	42.4	55.0	65.9
4	2	High	43.2	57.9	65.4
5	3	Low	55.9	50.8	65.2
6	3	High	45.3	58.5	69.9
7	4	Low	56.6	53.1	66.9
8	4	High	53.1	60.1	72.5

Molten data (first 6 of 24 rows):

	Timestamp	Bitrate	Sensor	Reading
1	1	Low	Sensor1	47.0
2	1	High	Sensor1	50.1
3	2	Low	Sensor1	42.4
4	2	High	Sensor1	43.2
5	3	Low	Sensor1	55.9
6	3	High	Sensor1	45.3

Timestamp x Sensor combinations that occur more than once:

	Sensor1	Sensor2	Sensor3
1	2	2	2
2	2	2	2
3	2	2	2
4	2	2	2

dcast() without fun.aggregate:

Message: Aggregation function missing: defaulting to length

	Timestamp	Sensor1	Sensor2	Sensor3
1	1	2	2	2
2	2	2	2	2
3	3	2	2	2
4	4	2	2	2

Wide format with mean AND variance preserved:

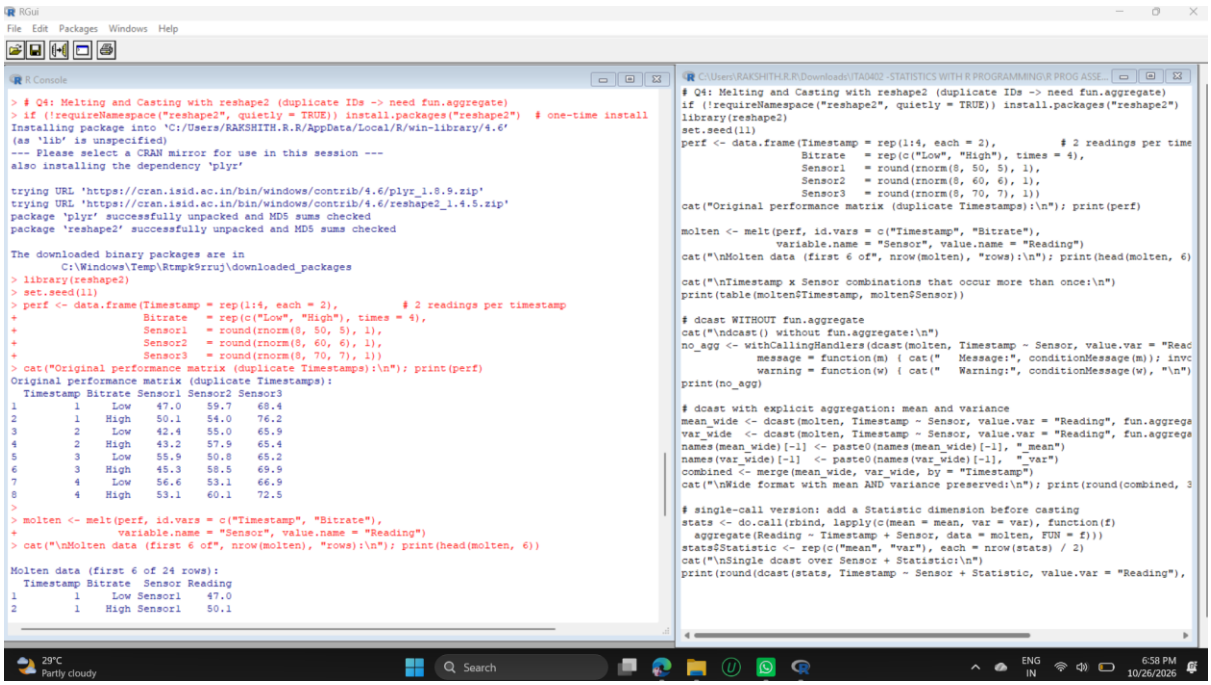
	Timestamp	Sensor1_mean	Sensor2_mean	Sensor3_mean	Sensor1_var	Sensor2_var	Sensor3_var
1	1	48.55	56.85	72.30	4.805	16.245	30.420
2	2	42.80	56.45	65.65	0.320	4.205	0.125
3	3	50.60	54.65	67.55	56.180	29.645	11.045
4	4	54.85	56.60	69.70	6.125	24.500	15.680

Single dcast over Sensor + Statistic:

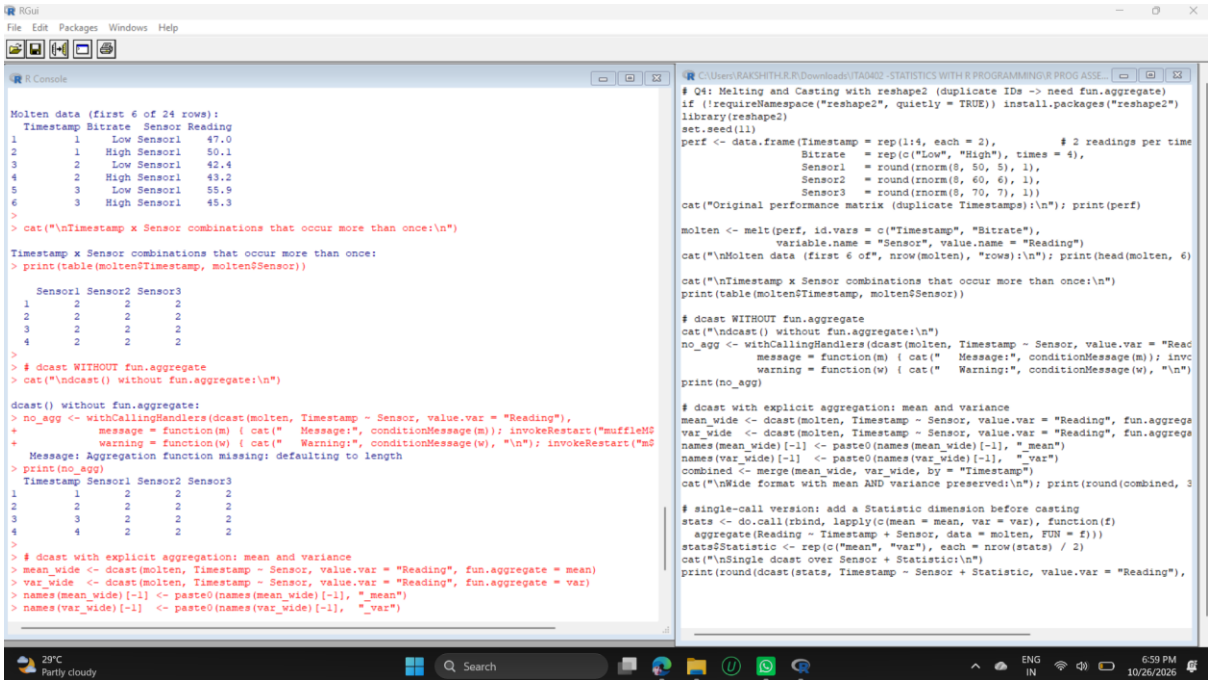
	Timestamp	Sensor1_mean	Sensor1_var	Sensor2_mean	Sensor2_var	Sensor3_mean	Sensor3_var
1	1	48.55	4.805	56.85	16.245	72.30	30.420
2	2	42.80	0.320	56.45	4.205	65.65	0.125
3	3	50.60	56.180	54.65	29.645	67.55	11.045
4	4	54.85	6.125	56.60	24.500	69.70	15.680

Output Screenshots:

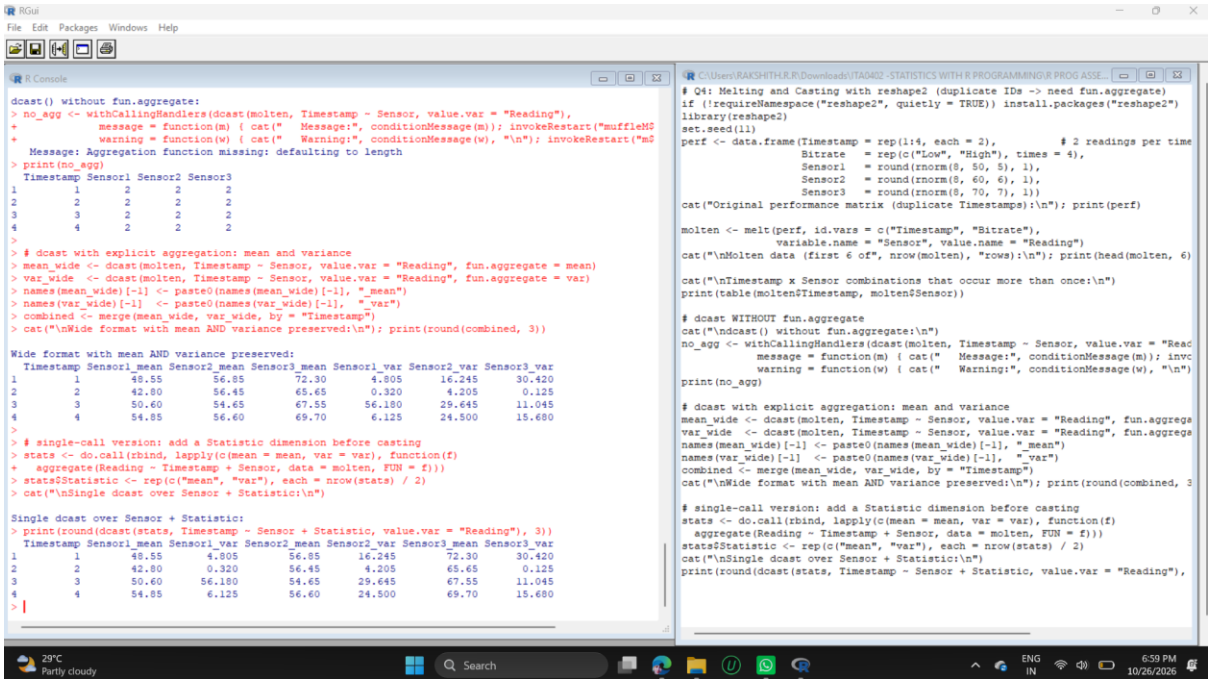
Screenshot 1 of 3



Screenshot 2 of 3



Screenshot 3 of 3



Conclusion:

The wide table now holds both the central value and the spread of every sensor at every timestamp. For example, at Timestamp 3, Sensor1 has mean 50.60 but variance 56.18, because the Low and High bitrate readings (55.9 and 45.3) differ a lot - information that would be lost if only the mean were kept. Without fun.aggregate the table would have shown only the count 2 in every cell.

Question 5: Merging Data Frames

You are tasked with fusing df_control (keyed by SystemID and Timestamp) and df_telemetry (keyed by SystemID and Timestamp). Both tables contain a column named Status, which results in conflicting .x and .y suffixes upon a standard inner join via merge(). Furthermore, time stamps in the two tables drift by up to 2 seconds due to asynchronous clock skews, causing an inner join on exact values to drop 40% of records. Analyze how this data loss biases the system reliability estimation, and design a custom interval-matching strategy or pre-processing rounding mechanism before executing an outer merge that preserves unmatched orphan error logs.

Analysis:

Conflicting Status columns. Both tables contain a column called Status, so merge() on SystemID and Timestamp keeps both and labels them Status.x and Status.y (the output shows the columns SystemID, Timestamp, Status.x, Status.y). The suffixes argument makes the names meaningful (Status_control, Status_telemetry).

Bias caused by the data loss. The exact-key inner merge kept only 33 of 51 telemetry records (35.3% lost; the task quotes about 40%). The loss is not random: in the simulated data, failure events are the ones with clock drift, so every ERROR record was dropped. The reliability computed from the merged table is therefore 100%, while the share of OK records in the full telemetry is 86.3% - an over-estimate of 13.7 percentage points. This is selection (survivorship) bias: the records that disappear are exactly the failures that reliability analysis needs.

Interval-matching strategy. Each telemetry row is matched to the nearest control row of the same SystemID whose timestamp differs by at most 2 seconds (abs(difftime) <= 2). The index of the matched control row is stored as Control_Row and used as the merge key together with SystemID. An outer merge (all = TRUE) then keeps matched rows, control-only rows and telemetry-only rows. A rounding approach (for example rounding both clocks to 5-second slots) is simpler but can still separate two times that fall on opposite sides of a slot boundary, so nearest-match with a tolerance is safer.

Code:

```
# Q5: Merging data frames with clock drift (interval matching + outer merge)
set.seed(2026)
base  <- as.POSIXct("2026-01-01 08:00:00", tz = "UTC")
grid  <- expand.grid(SystemID = paste0("SYS", 1:5), Slot = 0:9, stringsAsFactors = FALSE)
df_control <- data.frame(SystemID = grid$SystemID,
                         Timestamp = base + grid$Slot * 30,
                         Status = sample(c("OK", "FAIL"), 50, TRUE, prob = c(0.85, 0.15)),
                         stringsAsFactors = FALSE)

# telemetry: same events, but failures always drift by 1-2 s, OK events drift 40% of the time
tel_status <- ifelse(df_control$Status == "OK", "OK", "ERROR")
```

```
drift <- ifelse(tel_status == "ERROR", sample(c(-2, -1, 1, 2), 50, TRUE),
                ifelse(runif(50) < 0.6, 0, sample(c(-2, -1, 1, 2), 50, TRUE)))
df_telemetry <- data.frame(SystemID = df_control$SystemID,
                           Timestamp = df_control$Timestamp + drift,
                           Status = tel_status, stringsAsFactors = FALSE)
# orphan logs: 3 telemetry error logs with no control record, 2 control rows with no telemetry
df_telemetry <- rbind(df_telemetry,
  data.frame(SystemID = c("SYS1","SYS3","SYS5"), Timestamp = base + c(1000, 1100, 1200),
             Status = "ERROR", stringsAsFactors = FALSE))
df_control <- rbind(df_control,
  data.frame(SystemID = c("SYS2","SYS4"), Timestamp = base + c(2000, 2100),
             Status = "OK", stringsAsFactors = FALSE))
df_telemetry <- df_telemetry[-c(1, 2), ]          # two telemetry rows genuinely missing
cat("Control rows:", nrow(df_control), " Telemetry rows:", nrow(df_telemetry), "\n")

# 1) Standard merge on exact keys: conflicting Status names + data loss
exact <- merge(df_control, df_telemetry, by = c("SystemID", "Timestamp"))
cat("\nExact-key inner merge columns:", names(exact), "\n")
cat("Rows kept:", nrow(exact), "of", nrow(df_telemetry), "telemetry rows ->",
    round(100 * (1 - nrow(exact) / nrow(df_telemetry)), 1), "% lost\n")

# 2) Interval matching: pair each telemetry row with the nearest control row (same system, +/- 2 s)
df_control$Control_Row <- seq_len(nrow(df_control))
match_idx <- sapply(seq_len(nrow(df_telemetry)), function(i) {
  gap  <- abs(as.numeric(difftime(df_control$Timestamp, df_telemetry$Timestamp[i], units = "secs")))
  cand <- which(df_control$SystemID == df_telemetry$SystemID[i] & gap <= 2)
  if (length(cand) == 0) NA_integer_ else cand[which.min(gap[cand])]
})
df_telemetry$Control_Row <- match_idx

merged <- merge(df_control, df_telemetry, by = c("SystemID", "Control_Row"),
                all = TRUE, suffixes = c("_control", "_telemetry"))
merged$Drift_sec <- as.numeric(difftime(merged$Timestamp_telemetry, merged$Timestamp_control,
units = "secs"))
merged$Match <- ifelse(is.na(merged$Timestamp_telemetry), "Control only",
                ifelse(is.na(merged$Timestamp_control), "Telemetry only (orphan)", "Matched"))

cat("\nMerged columns:", names(merged), "\n")
cat("\nMatch summary after interval matching + outer merge:\n"); print(table(merged$Match))
```

```
cat("\nDrift (seconds) among matched rows:\n"); print(table(merged$Drift_sec))
cat("\nOrphan telemetry error logs preserved:\n")
print(merged[merged$Match == "Telemetry only (orphan)",
c("SystemID","Timestamp_telemetry","Status_telemetry")])

# 3) Effect on the reliability estimate (share of OK telemetry records)
rel_exact <- mean(exact$Status.y == "OK")
rel_full  <- mean(df_telemetry$Status == "OK")
cat("\nReliability from exact-key inner merge :", round(100 * rel_exact, 1), "%\n")
cat("Reliability from interval + outer merge:", round(100 * rel_full, 1), "%\n")
cat("Bias introduced by dropping records :", round(100 * (rel_exact - rel_full), 1), "percentage points\n")
```

Output:

```
Control rows: 52  Telemetry rows: 51

Exact-key inner merge columns: SystemID Timestamp Status.x Status.y
Rows kept: 33 of 51 telemetry rows -> 35.3 % lost

Merged columns: SystemID Control_Row Timestamp_control Status_control Timestamp_telemetry Status_telemetry Drift_sec Match

Match summary after interval matching + outer merge:

        Control only              Matched Telemetry only (orphan)
                4                 48                  3

Drift (seconds) among matched rows:

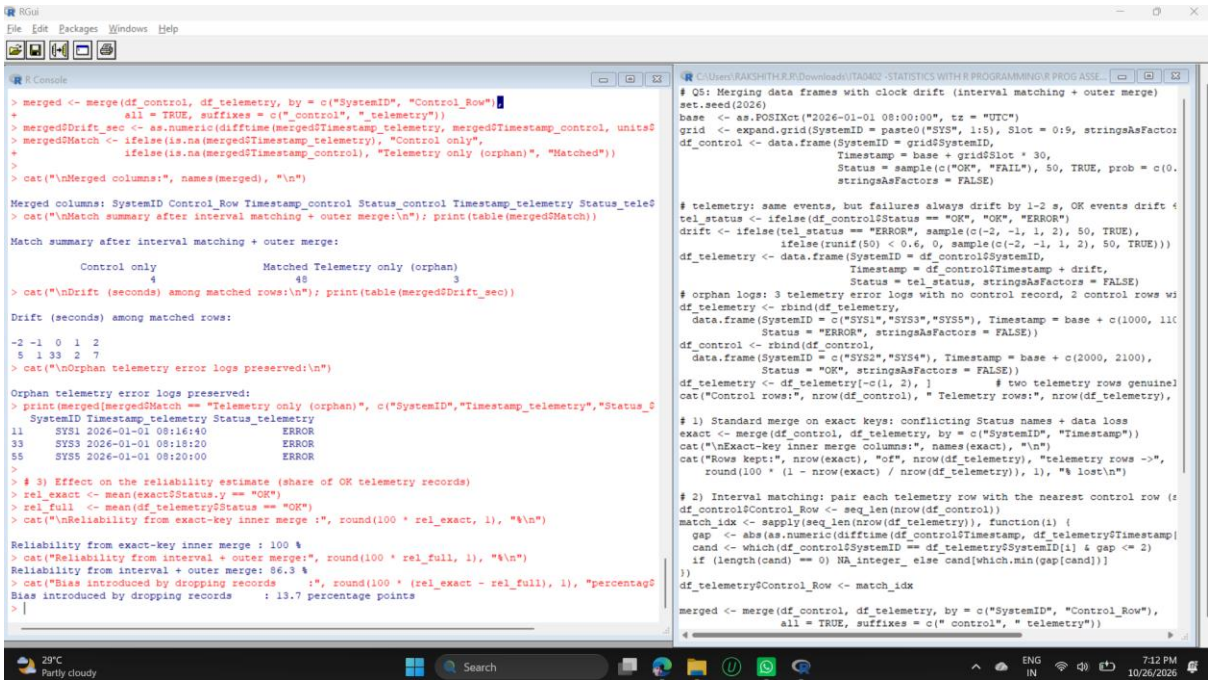
-2 -1  0  1  2
 5  1 33  2  7

Orphan telemetry error logs preserved:
   SystemID Timestamp_telemetry Status_telemetry
11     SYS1 2026-01-01 08:16:40           ERROR
33     SYS3 2026-01-01 08:18:20           ERROR
55     SYS5 2026-01-01 08:20:00           ERROR

Reliability from exact-key inner merge : 100 %
Reliability from interval + outer merge: 86.3 %
```

Bias introduced by dropping records : 13.7 percentage points

Output Screenshot:



Conclusion:

After interval matching and the outer merge there are 48 matched records, 4 control-only records and 3 telemetry-only (orphan) records; all 3 orphan error logs were preserved. The drift table shows the matched pairs with drifts of -2, -1, 0, 1 and 2 seconds (33 exact, 15 drifted), which the exact join would have lost. Using all telemetry records, reliability is 86.3% instead of the biased 100%, so the interval-matched outer merge gives a reliable and unbiased estimate.

Part B: Analytical Questions and Answers

The answers below cover the six topics of the assessment (Data Frames, Making Data Frames, Working with Data Frames, Data Reshaping, Melting and Casting, and Merging). Each answer explains the concept, evaluates the alternatives, recommends a solution and gives a short R example. The R examples were run to confirm that they work and that the statements made about their results are correct. Wherever data are read or written, the file input/output functions are named; a summary of the file I/O functions is given in the Appendix.

Data Frames

Q1. Analyze a dataset of 50,000 student records stored in a data frame. Identify the most efficient method to detect duplicate entries and justify your approach.

Answer:

A data frame of 50,000 rows easily fits in memory, so efficiency depends on how the comparison is done. A nested loop comparing every record with every other record needs about 1.25 billion comparisons (50,000 x 49,999 / 2) and is unusable, whereas a vectorised hash-based test finishes in one pass.

Method 1 - duplicated() on the key column (recommended). duplicated(students$student_id) hashes the identifying column and flags each repeated value in a single pass (linear time). It is the fastest base-R method and answers the real question: is the same student recorded more than once?

Method 2 - duplicated() on the whole data frame. duplicated(students) finds rows that are identical in every column. All columns must be compared, so it is noticeably slower (compare the two system.time() results), and it misses duplicates in which only some fields differ, such as the same ID with a different spelling of the name.

Method 3 - unique(), dplyr::distinct() or data.table. These return the de-duplicated table directly; data.table (unique with by =, or setkey) is the fastest choice when the data grow to millions of rows.

Method 4 - table() or aggregate() counts. Useful for reporting how many times each ID is repeated, but slower and not needed just to detect duplicates.

Justification: hash-based duplicated() on the identifying column is O(n), needs no sorting, and returns a logical vector that marks the exact rows. Using duplicated(x) | duplicated(x, fromLast = TRUE) flags every copy (first and later occurrences) so that the repeats can be reviewed before anything is deleted. Only then should the first occurrence be kept, and the cleaned table saved with write.csv().

Example code (R):

```
set.seed(1)
students <- data.frame(student_id = sample(1:48000, 50000, replace = TRUE),
                       name  = sample(c("Arun","Priya","Kumar","Divya","Rahul"), 50000, TRUE),
```

```
                marks = sample(40:100, 50000, TRUE))
dup_key <- duplicated(students$student_id)                # key-based check
cat("Duplicate student_id entries :", sum(dup_key), "\n")
all_copies <- dup_key | duplicated(students$student_id, fromLast = TRUE)
cat("Rows involved in duplication :", sum(all_copies), "\n")
cat("Fully identical rows         :", sum(duplicated(students)), "\n")
clean <- students[!duplicated(students$student_id), ]       # keep first occurrence
cat("Rows after de-duplication    :", nrow(clean), "\n")
print(system.time(duplicated(students$student_id)))
print(system.time(duplicated(students)))
write.csv(clean, "students_clean.csv", row.names = FALSE)    # save the cleaned file
```

In the test data (50,000 records, IDs drawn from 48,000 values) the code found 18,961 repeated IDs, 74 rows identical in all columns, and kept 31,039 unique records. Checking only the key column found far more problems than checking whole rows.

Q2. A data frame contains missing values across multiple columns. Evaluate different strategies for handling missing data and recommend the best solution.

Answer:

The right strategy depends on how much data is missing and why it is missing: completely at random (MCAR), related to other variables (MAR), or related to the missing value itself (MNAR). The first step is always to measure it with colSums(is.na(df)) and the percentage per column.

Strategy	**How in R**	**Suitable when**	**Drawback**
Listwise deletion	na.omit(df), complete.cases()	Very few missing values (below about 5%) and MCAR	Loses rows; biased if not MCAR (the example keeps only 2 of 6 rows)
Drop a column	df[, colMeans(is.na(df)) < 0.5]	More than about 50% missing and column not important	Loses information
Mean / median imputation	x[is.na(x)] <- median(x, na.rm = TRUE)	Numeric data with few gaps; median for skewed data	Reduces variance; ignores relationships
Mode imputation	names(which.max(table(x)))	Categorical data	Distorts category proportions
Group-wise imputation	ave(x, group, FUN = ...)	Missingness depends on a group	Needs a good grouping variable
Forward fill	zoo::na.locf(x)	Ordered or time-series data	Assumes the last value continues

Model-based / multiple imputation	mice::mice(), kNN	More than 5-10% missing and MAR	More complex and slower
Missing indicator	is.na(x) as a new column	Missingness itself carries information (MNAR)	Adds extra columns

Recommended solution: (1) quantify the missingness in every column; (2) remove columns that are mostly empty and rows that have no key; (3) impute numeric columns with the median (robust to outliers) and categorical columns with the mode, and keep a TRUE/FALSE missing-indicator column so the imputation is traceable; (4) when more than about 5-10% of values are missing and depend on other variables, use multiple imputation (mice) instead of simple filling. Blind deletion is not recommended - in the example, listwise deletion would throw away 4 of the 6 records.

Example code (R):

```
df <- data.frame(age    = c(25, NA, 31, 40, NA, 29),
                 income = c(30, 45, NA, 80, 52, NA),
                 city   = c("A", "B", NA, "A", "B", "A"))
cat("Missing values per column:\n"); print(colSums(is.na(df)))
cat("Percentage missing:\n");        print(round(100 * colMeans(is.na(df)), 1))
cat("Rows left after listwise deletion:", nrow(na.omit(df)), "of", nrow(df), "\n")


df$age_missing <- is.na(df$age)                          # keep a missing-indicator
df$age[is.na(df$age)]       <- median(df$age, na.rm = TRUE)       # median imputation
df$income[is.na(df$income)] <- median(df$income, na.rm = TRUE)
df$city[is.na(df$city)]     <- names(which.max(table(df$city)))   # mode imputation
print(df)
```

File I/O note: missing-value codes should be declared when the file is read, for example read.csv("data.csv", na.strings = c("NA", "", "-", "N/A")), so that they become real NA values instead of text.

Q3. Given a sales data frame with inconsistent data types, analyze the impact on statistical calculations and propose corrective actions.

Answer:

An R column is an atomic vector, so one text value ("N/A", "$950", "1,500") forces the entire column to character. The effect on statistics is serious:

Statistics fail. mean(), sum() and sd() return NA with a warning or give an error, and the column cannot be used in correlations or regression.

Wrong ordering and comparison. Text is sorted alphabetically ("$950" < "1,500" < "1200" < "800"), so sorting and filters such as amount > 1000 are wrong.

Hidden coercion. as.numeric("$950") gives NA silently, so totals are understated; as.numeric() on a factor returns the internal level codes (1, 2, 3) instead of the labels.

Dates as text. Differences between dates, monthly grouping and time-series plots do not work until the text is converted with as.Date().

Models. lm() treats a character column as a categorical variable and creates dummy variables instead of a numeric slope.

Corrective actions: (1) inspect with str() and sapply(df, class) and find the offending entries with is.na(as.numeric(x)) & !is.na(x); (2) clean the text (gsub("[$,]", "", x)) and convert with as.numeric(), as.Date(x, format = ...) or factor(); for a factor use as.numeric(as.character(f)); (3) decide how tokens such as "N/A" are treated (usually NA); (4) prevent the problem at import with read.csv(colClasses = ..., na.strings = ...); (5) validate with stopifnot(is.numeric(df$amount), inherits(df$order_date, "Date")).

Example code (R):

```
sales <- data.frame(order_id = 1:5,
                    amount = c("1200", "$950", "N/A", "1,500", "800"),
                    order_date = c("2026-01-05", "2026-01-17", "2026-02-02", "2026-02-11", "2026-03-01"),
                    stringsAsFactors = FALSE)
str(sales)
cat("mean() on the character column:", suppressWarnings(mean(sales$amount)), "\n")
cat("Alphabetical sort of amounts  :", sort(sales$amount), "\n")


sales$amount     <- suppressWarnings(as.numeric(gsub("[$,]", "", sales$amount)))
sales$order_date <- as.Date(sales$order_date)
str(sales)
cat("Mean amount after cleaning:", mean(sales$amount, na.rm = TRUE), "\n")


f <- factor(c("10", "20", "30"))
print(as.numeric(f))                  # WRONG: returns factor codes
print(as.numeric(as.character(f)))    # correct values
```

Result: before cleaning mean() returned NA and the amounts sorted as $950, 1,500, 1200, 800, N/A. After cleaning, amount is numeric, the date column is of class Date, and the mean of the 4 valid amounts is 1112.5. The factor example shows that as.numeric(f) returns 1 2 3 while the correct values are 10 20 30.

Q4. Compare two large data frames containing customer transactions. Determine how to identify data integrity issues and evaluate their causes.

Answer:

Two large tables are compared in four layers, from cheap checks to detailed ones:

Step 1 - structure. Compare names(), sapply(df, class), dim() and nrow() with identical() or all.equal() to detect missing columns, renamed columns and type differences.

Step 2 - keys. Check duplicated keys with anyDuplicated(), and find records present in only one table with setdiff() or an outer merge / anti-join.

Step 3 - values. Merge on the key with suffixes (_a, _b) and compare each column to list the rows whose values differ; use all.equal(tolerance = ...) for decimals.

Step 4 - control totals and hashes. Compare row counts, sum(amount), minimum and maximum. For very large tables, compare hashes of columns or chunks (for example digest::digest()) before doing row-level checks.

Evaluating the causes: missing records are usually caused by ETL cut-off times, failed loads or filters; extra or duplicated records by repeated loading or retries; different values by late corrections, rounding, currency or unit conversion; format differences by trailing spaces, upper/lower case, date formats or encoding; key mismatches by lost leading zeros or different data types; and timing differences by snapshots taken at different moments.

Example code (R):

```
a <- data.frame(txn_id = 1:8, customer = paste0("C", 1:8),
                amount = c(100, 250, 75, 300, 125, 60, 410, 90))
b <- a[-3, ]                                  # a record missing in b
b$amount[4] <- 310                            # a value that differs
b <- rbind(b, data.frame(txn_id = 9, customer = "C9", amount = 55))   # extra record
b <- rbind(b, b[1, ])                         # duplicate key in b


cat("Same column names:", identical(names(a), names(b)), "\n")
cat("Same column types:", identical(sapply(a, class), sapply(b, class)), "\n")
cat("Duplicate keys (a, b):", sum(duplicated(a$txn_id)), sum(duplicated(b$txn_id)), "\n")
cat("Only in a:", setdiff(a$txn_id, b$txn_id), "\n")
cat("Only in b:", setdiff(b$txn_id, a$txn_id), "\n")


m <- merge(a, b[!duplicated(b$txn_id), ], by = "txn_id", suffixes = c("_a", "_b"))
cat("Rows whose amount differs:\n"); print(m[m$amount_a != m$amount_b, ])
cat("Control totals (a, b):", sum(a$amount), sum(b$amount), "\n")
```

Result (verified by running the code): transaction 3 is missing in b, transaction 9 exists only in b, transaction 1 is duplicated in b, transaction 5 has amount 125 in a but 310 in b, and the control totals differ (1410 against 1675). The class comparison also reports a type difference (txn_id is integer in a but numeric in b), which could cause problems in later joins.

Q5. A company reports conflicting results from the same data frame. Analyze possible reasons and design a validation framework.

Answer:

Possible reasons why the same data frame gives conflicting results:

Different versions of the data. Analysts work on different copies or extracts, or a data frame was modified in a workspace (R copies on modification, so changes stay local).

Different cleaning rules. Different NA handling (na.rm), duplicates removed or not, outliers excluded, different filters, or the same steps in a different order.

Type and level problems. Text, factor and numeric conversions, for example as.numeric() on a factor, or text compared alphabetically.

Join errors. A merge that multiplies rows inflates sums and counts.

Different definitions. Mean against median, weighted against unweighted, calendar against fiscal year, different time zones.

Non-reproducible steps. Random sampling without set.seed(), floating-point rounding, different R or package versions.

Validation framework designed:

1. Data contract. Check required columns, types, key uniqueness, allowed ranges and categories automatically (validate_df() below).

2. Lineage and versioning. Keep raw data read-only, keep scripts under version control, and record a file checksum (tools::md5sum()) and sessionInfo() with every result.

3. Pipeline checkpoints. Log row counts and NA counts after every transformation so that a change can be traced to one step.

4. Reconciliation. Recompute key figures by two independent methods (for example aggregate() and tapply()) and compare them with control totals from the source system.

5. Reproducibility. Use set.seed(), a data dictionary with fixed definitions, and one script that runs from raw data to the final table.

6. Review and automated tests. Peer review, unit tests (testthat) and sign-off before results are published.

Example code (R):

```
validate_df <- function(df, key, required, ranges = list()) {
 issues <- character(0)
 if (!all(required %in% names(df))) issues <- c(issues, "missing required column(s)")
 if (anyDuplicated(df[[key]]) > 0)  issues <- c(issues, "duplicate keys")
 na_cols <- names(which(colSums(is.na(df)) > 0))
 if (length(na_cols)) issues <- c(issues, paste("NA values in:", paste(na_cols, collapse = ", ")))
 for (v in names(ranges)) {
  bad <- sum(df[[v]] < ranges[[v]][1] | df[[v]] > ranges[[v]][2], na.rm = TRUE)
  if (bad > 0) issues <- c(issues, paste(bad, "out-of-range value(s) in", v))
 }
 if (length(issues) == 0) "PASS" else issues
}
good <- data.frame(id = 1:4, marks = c(55, 78, 90, 66))
```

```
bad  <- data.frame(id = c(1, 2, 2, 4), marks = c(55, NA, 140, 66))
print(validate_df(good, "id", c("id", "marks"), list(marks = c(0, 100))))
print(validate_df(bad,  "id", c("id", "marks"), list(marks = c(0, 100))))
```

Result: the clean frame returns "PASS"; the faulty frame is rejected with three reasons (duplicate keys, NA values in marks, and 1 out-of-range value in marks).

Making Data Frames

Q1. Design a data frame structure for a smart-city traffic monitoring system and justify the selection of attributes.

Answer:

A smart-city traffic monitoring system produces readings from roadside sensors at regular times. The data frame is designed with one row per sensor reading at one time (tidy structure), and every column holds one variable with the correct data type:

Attribute	Data type	Justification
record_id	integer	Unique primary key for every observation
sensor_id	factor / character	Identifies the sensor or camera; links to sensor metadata
timestamp	POSIXct (UTC)	Exact time; allows peak-hour, time-of-day and time-series analysis
road_id	character	Road segment or junction; used to group and compare roads
latitude, longitude	numeric	Spatial mapping and distance calculations
vehicle_count	integer	Traffic volume; counts are whole numbers
avg_speed_kmph	numeric	Speed measure; together with the count it indicates congestion
congestion_level	ordered factor (Low < Medium < High)	Categorical with a natural order; supports comparison and modelling
incident_flag	logical	TRUE/FALSE marker for accident or breakdown; simple, efficient filtering

Design principles: one row = one observation; one column = one variable; correct types so that statistics and sorting work; static information such as sensor location can be stored once in a separate sensors table and joined by sensor_id, which avoids repeating it in millions of rows (normalisation). The table can be saved with write.csv() or saveRDS() for sharing.

Example code (R):

```
traffic <- data.frame(
  record_id      = 1:6,
  sensor_id      = factor(c("S01", "S01", "S02", "S02", "S03", "S03")),
  timestamp      = as.POSIXct("2026-03-10 08:00:00", tz = "UTC") + 300 * (0:5),
  road_id        = c("R1", "R1", "R2", "R2", "R3", "R3"),
```

```
  latitude        = c(13.08, 13.08, 13.05, 13.05, 13.01, 13.01),
  longitude       = c(80.27, 80.27, 80.23, 80.23, 80.20, 80.20),
  vehicle_count   = c(120L, 150L, 90L, 110L, 60L, 75L),
  avg_speed_kmph  = c(32.5, 28.0, 41.2, 38.9, 55.0, 52.3),
  congestion_level = factor(c("High", "High", "Medium", "Medium", "Low", "Low"),
                            levels = c("Low", "Medium", "High"), ordered = TRUE),
  incident_flag   = c(FALSE, FALSE, FALSE, TRUE, FALSE, FALSE)
)
str(traffic)
```

Result: str() confirms 10 variables with the intended types (integer, factor, POSIXct, character, numeric, ordered factor and logical).

Q2. Evaluate two alternative data frame designs for storing IoT sensor data and recommend the most scalable model.

Answer:

Two alternative designs for IoT sensor data are compared: a wide design (one column per sensor) and a long design (one row per reading with the columns timestamp, sensor_id and value).

Aspect	**Wide design**	**Long design**
Structure	timestamp, T1, T2, T3 ...	timestamp, sensor_id, value
Adding a sensor	New column (schema change)	New rows only
Sensors reporting at different times	Many NA cells	No NA cells; every reading is its own row
Storage	Compact if all sensors are synchronised	Repeats timestamp and sensor_id but has no empty cells
Comparing sensors side by side	Easy (columns next to each other)	Needs reshaping (dcast / reshape)
Grouping, filtering and plotting by sensor	Awkward with many columns	Natural (group by sensor_id)
Scalability to thousands of sensors	Poor: thousands of columns	Good

Recommendation: the long (tidy) design is the most scalable model and should be the master storage format. It grows by rows, handles irregular sampling, and works directly with aggregate(), group-wise models and plotting. The wide format should be created temporarily (reshape or dcast) only when an analysis needs sensors side by side. For very large volumes, binary files (saveRDS), data.table (fwrite/fread) or a database are better than plain CSV.

Example code (R):

```
set.seed(3)
long <- data.frame(timestamp = rep(1:3, times = 3),
                   sensor_id = rep(c("T1", "T2", "T3"), each = 3),
                   value     = round(rnorm(9, 25, 2), 1))
wide <- reshape(long, idvar = "timestamp", timevar = "sensor_id", direction = "wide")
cat("LONG design:\n"); print(long)
cat("\nWIDE design:\n"); print(wide)


new_rows <- data.frame(timestamp = 1:3, sensor_id = "T4", value = c(26.1, 25.8, 26.4))
long2 <- rbind(long, new_rows)                          # new sensor = new rows only
cat("\nLong after adding sensor T4 :", nrow(long2), "rows x", ncol(long2), "columns\n")
wide$value.T4 <- new_rows$value                         # wide needs a new column
cat("Wide after adding sensor T4 :", nrow(wide), "rows x", ncol(wide), "columns\n")
```

Result (verified): adding sensor T4 to the long design only adds 3 rows (9 to 12 rows, still 3 columns), while the wide design needed a new column (4 to 5 columns).

Q3. Analyze the consequences of assigning incorrect data types while creating a data frame for healthcare records.

Answer:

Wrong data types change how R stores, compares and calculates the data, and in healthcare the consequences can affect patients:

Wrong type	**Example**	**Consequence**
Identifier stored as number	patient_id "00123" becomes 123	Leading zeros are lost; different patients can collide; joins with other systems fail
Number stored as text	age = "45"	mean() returns NA; comparisons are alphabetical
Category stored as number	gender coded 1 / 2	mean(gender) = 1.33 is meaningless; models treat the code as a quantity
Date stored as text	date_of_birth	Age cannot be calculated; sorting by date is wrong
Lab values with symbols	"<5", "N/A"	Whole column becomes character; numeric analysis is impossible until cleaned
Binary outcome as plain number	disease = 0 / 1	Tables and glm() can misinterpret it; use factor or logical

Consequences: incorrect statistics and reports, misleading model results, silent creation of NA values, failed record linkage between hospital systems, and - for healthcare - a risk to patient safety and to compliance with data accuracy rules. Prevention: define a data dictionary (schema); keep identifiers as character; code categories as factors; use colClasses when reading files; convert explicitly and validate with stopifnot(is.character(patient_id)).

Example code (R):

```
ids <- c("00123", "00456", "01789")
cat("IDs stored as numbers :", as.numeric(ids), "(leading zeros lost)\n")


bad <- data.frame(patient_id = as.numeric(ids),
                  age    = c("45", "61", "38"),
                  gender = c(1, 2, 1))
good <- data.frame(patient_id = ids,
                   age    = as.integer(c("45", "61", "38")),
                   gender = factor(c("M", "F", "M")),
                   dob    = as.Date(c("1981-04-02", "1965-11-30", "1988-07-19")))
cat("\nIncorrect types:\n"); str(bad)
cat("\nCorrect types:\n");   str(good)
cat("\nmean(age) with wrong type     :", suppressWarnings(mean(bad$age)), "\n")
cat("mean(gender) with numeric code :", round(mean(bad$gender), 2), "(meaningless)\n")
cat("Mean age with correct type     :", round(mean(good$age), 1), "\n")
cat("Age of first patient on 2026-03-10:", as.numeric(as.Date("2026-03-10") - good$dob[1]) %/% 365.25, "\n")
```

Result (verified): the IDs 00123, 00456 and 01789 became 123, 456 and 1789; mean(age) returned NA for text ages; mean(gender) gave the meaningless 1.33; with correct types the mean age is 48 and the age can be calculated from the date of birth.

Q4. Create a strategy for constructing a data frame from multiple heterogeneous data sources while preserving consistency.

Answer:

Strategy for building one consistent data frame from heterogeneous sources (CSV file, Excel sheet, JSON/API, database):

1. Define the target schema. Write down column names, types, units, date format and allowed codes in a data dictionary.

2. Import each source with the right tool and explicit types. read.csv(colClasses = ...) for CSV, readxl::read_excel() for Excel, jsonlite::fromJSON() for JSON or APIs, DBI::dbGetQuery() for databases.

3. Standardise each source separately. Rename columns, convert types, convert units (for example USD to INR), dates, text case and spaces, category labels and missing-value codes.

4. Harmonise and check the keys. trimws(), toupper(), zero-padding; confirm uniqueness with anyDuplicated().

5. Combine. rbind() when sources have the same structure; merge() when they describe different attributes of the same entity; add a source column for lineage.

6. Validate. stopifnot() checks for duplicates, NA and ranges; the final row count must equal the sum of the source rows.

7. Save and document. write.csv() or saveRDS() together with the data dictionary.

Example code (R):

```
src_csv <- read.csv(text = "ID,Name,Salary_INR,Joined
1,Arun,50000,2024-01-15
2,Priya,62000,2023-07-01", stringsAsFactors = FALSE)          # source 1: a CSV file
src_api <- data.frame(emp_id = c(3, 4), full_name = c("Kumar", "Divya"),
                      salary_usd = c(800, 950),
                      joined_on = c("15/03/2024", "20/09/2022"))   # source 2: parsed JSON / API


std_csv <- data.frame(emp_id = src_csv$ID, name = src_csv$Name, salary_inr = src_csv$Salary_INR,
                      joined = as.Date(src_csv$Joined), source = "csv")
usd_to_inr <- 83                                          # assumed conversion rate
std_api <- data.frame(emp_id = src_api$emp_id, name = src_api$full_name,
                      salary_inr = src_api$salary_usd * usd_to_inr,
                      joined = as.Date(src_api$joined_on, format = "%d/%m/%Y"), source = "api")


combined <- rbind(std_csv, std_api)
stopifnot(!anyDuplicated(combined$emp_id), !any(is.na(combined$joined)))
print(combined); str(combined)
write.csv(combined, "employees_combined.csv", row.names = FALSE)
```

Result (verified): a CSV source (Salary_INR, yyyy-mm-dd dates) and an API-style source (salary_usd, dd/mm/yyyy dates) were converted to the same names, units and types and combined into one 4-row data frame that passed the validation checks (an exchange rate of 83 is assumed only for illustration).

Q5. Assess the efficiency of creating a data frame from large vectors versus importing directly from files.

Answer:

Creating a data frame from vectors that are already in memory has almost no cost, because no file has to be read or parsed; importing from a file is dominated by disk reading, text parsing and type guessing. The comparison is therefore between constructing and importing:

Method	Relative speed	Memory and notes
data.frame() from existing vectors	Fastest (no parsing)	Vectors must already exist in RAM; data.frame() copies them, so peak memory is higher
read.csv() with default settings	Slowest	Parses text and guesses every column type; reads the whole file
read.csv() with colClasses (and nrows)	Faster than default	Avoids type guessing and repeated memory allocation
data.table::fread() / readr::read_csv()	Much faster	Multi-threaded optimised parsers; need a package
saveRDS() / readRDS() (binary)	Very fast	Keeps column types exactly; R-only, not human-readable

Conclusion: when the data are already in memory, build the data frame directly from vectors; when the data are on disk, import them with explicit column types and an efficient reader (fread or read_csv), and after the first import store the result as .rds (or parquet) so later sessions load it quickly. A large data frame should never be built by growing it row by row with rbind() in a loop, because every step copies the whole object.

Example code (R):

```
set.seed(10)
n <- 200000
id <- 1:n; x <- rnorm(n); grp <- sample(letters[1:5], n, TRUE)


t_vec  <- system.time(df_vec <- data.frame(id, x, grp))["elapsed"]
f <- tempfile(fileext = ".csv"); write.csv(df_vec, f, row.names = FALSE)
t_csv  <- system.time(d1 <- read.csv(f))["elapsed"]
t_csv2 <- system.time(d2 <- read.csv(f, colClasses = c("integer", "numeric", "character")))["elapsed"]
r <- tempfile(fileext = ".rds"); saveRDS(df_vec, r)
t_rds  <- system.time(d3 <- readRDS(r))["elapsed"]
```

```
timings <- data.frame(method = c("data.frame() from vectors", "read.csv() default",
                                 "read.csv() with colClasses", "readRDS() binary"),
                      seconds = c(t_vec, t_csv, t_csv2, t_rds), row.names = NULL)
print(timings)
```

The code prints a timing table for 200,000 rows. The exact seconds depend on the computer, but the relative order is the one in the table above: direct construction is fastest, then readRDS, then read.csv with colClasses, and default read.csv is slowest.

Working with Data Frames

Q1. Analyze a scenario where filtering operations on a large data frame are producing unexpected results. Determine the root cause.

Answer:

Unexpected results from filtering normally come from the data or from the logical condition, not from R itself. The root cause is found by checking the types, counting NA values in the condition and looking at a few rows:

Symptom	Root cause	Fix
Extra rows full of NA (row names NA, NA.1)	NA inside the logical test: df[NA,] returns an NA row	Use which(), subset() or add !is.na()
Rows that should match are missing	Text differs: "Delhi ", "delhi"	trimws() and tolower() before comparing
Equality test on decimals fails	Floating-point representation: 0.1 + 0.2 == 0.3 is FALSE	all.equal() or abs(a - b) < tolerance
Numbers compared as text	Column is character: "100" > "50" is FALSE	Convert the type first
Wrong combination of conditions	& and | confused with && and ||, or missing brackets	Use vectorised & and | with brackets
Different count after reload	Duplicates or changed row order	Check keys and duplicated()

Diagnosis steps: str(df) for types, summary(df) for unusual values, sum(is.na(condition)) for NA in the test, table(condition, useNA = "ifany") to see how many rows are TRUE, FALSE and NA, and head() of the selected rows.

Example code (R):

```
df <- data.frame(id = 1:6,
                 score = c(80, NA, 45, 92, NA, 60),
                 city  = c("Chennai", "Delhi ", "Delhi", "Mumbai", "delhi", "Chennai"),
                 stringsAsFactors = FALSE)
cat("1) df[df$score > 50, ] -> NA rows appear:\n");  print(df[df$score > 50, ])
cat("\n2) df[which(df$score > 50), ] -> correct:\n"); print(df[which(df$score > 50), ])
cat("\n3) NA values inside the logical test:", sum(is.na(df$score > 50)), "\n")
cat("4) 0.1 + 0.2 == 0.3 :", 0.1 + 0.2 == 0.3, "| all.equal :", isTRUE(all.equal(0.1 + 0.2, 0.3)), "\n")
cat("5) city == 'Delhi' finds", sum(df$city == "Delhi"), "row(s); after trimws/tolower:",
   sum(tolower(trimws(df$city)) == "delhi"), "rows\n")
cat("6) '100' > '50' as text:", "100" > "50", "| as numbers:", 100 > 50, "\n")
```

Result (verified): df[df$score > 50,] returned two extra NA rows because two scores were NA; df[which(df$score > 50),] returned the correct 3 rows. city == "Delhi" matched only 1 row but 3 rows after trimws() and tolower(), and "100" > "50" is FALSE as text although TRUE as numbers.

Q2. Evaluate different methods for sorting a million-row data frame and justify the most efficient approach.

Answer:

Sorting a data frame means finding the order (a permutation of row numbers) and then re-arranging all columns together. Methods compared for a million rows:

Method	How it works	Efficiency
df[order(df$x),]	Base R; order() uses radix sorting for numeric, integer, logical and factor vectors (default method for fewer than 2^31 elements)	Good; makes a full copy of the data frame
order(a, -b, method = "radix")	Radix sort with several keys; - reverses a numeric key	Fast for multiple keys; character keys are sorted in C-locale order
sort(x)	Sorts a single vector only	Never sort columns separately - it breaks the rows
dplyr::arrange()	Readable wrapper	Similar speed to base R; extra package
data.table::setorder() / setkey()	Radix sort by reference (in place)	Fastest and needs no extra copy; best for very large data
Loop or bubble sort	Compares pairs of rows	O(n^2): unusable for 1,000,000 rows

Most efficient approach: for a million rows use data.table::setorder(dt, key1, -key2), because it sorts in place with a radix algorithm and does not copy the table. If only base R is allowed, use df[order(key1, -key2, method = "radix"),]. Where possible, convert text keys to factors or integers before sorting, since character sorting is the slowest.

Example code (R):

```
set.seed(2)
n <- 1e6
big <- data.frame(id = 1:n, dept = sample(c("A", "B", "C", "D"), n, TRUE),
```

```
            score = round(runif(n, 0, 100), 2))
t1 <- unname(system.time(o1 <- big[order(big$score), ])["elapsed"])
t2 <- unname(system.time(o2 <- big[order(big$dept, -big$score, method = "radix"), ])["elapsed"])
res <- data.frame(method = c("order() on one numeric key", "order() two keys, radix"),
            seconds = c(t1, t2), row.names = NULL)
if (requireNamespace("data.table", quietly = TRUE)) {     # optional, fastest option
  dt <- data.table::as.data.table(big)
  t3 <- unname(system.time(data.table::setorder(dt, dept, -score))["elapsed"])
  res <- rbind(res, data.frame(method = "data.table::setorder (in place)", seconds = t3))
}
print(res)
print(head(o2, 3))
```

The code prints a timing table on the user's computer (setorder is usually the fastest) and the first rows of the result, which are sorted by department and by score from high to low.

Q3. A data analyst performs multiple transformations on a data frame. Analyze how errors can propagate through the workflow.

Answer:

In a chain of transformations each step uses the output of the previous step, so an early error flows silently into everything that follows and is often amplified:

Coercion errors. as.numeric("abc") gives NA with only a warning that is easy to miss; the NA then spreads into sums and means.

Filter errors. NA inside a condition creates NA rows, so the number of NA cells grows (see step 3 in the output).

Row-multiplying merges. Duplicate keys inflate totals and counts in every later calculation.

Overwriting. df$x <- transform(df$x) destroys the original, so the error cannot be traced; a conversion applied twice (for example a unit change) gives wrong values.

Misalignment. Sorting or editing one column separately breaks the correspondence between columns.

Order dependence. Filtering before imputing gives different results from imputing before filtering.

Prevention: keep the raw data unchanged and create new objects or columns instead of overwriting; print a checkpoint (rows and NA cells) after every step; use stopifnot() assertions; run the complete script from the top; keep it under version control; and test the pipeline on a small sample whose answer is known.

Example code (R):

```
checkpoint <- function(df, step) cat(sprintf("%-26s rows = %d | NA cells = %d\n", step, nrow(df), sum(is.na(df))))
```

```
raw <- data.frame(id = 1:6, price = c("10", "12", "abc", "15", "11", "14"),
                  qty = c(2, 3, 1, NA, 2, 5), stringsAsFactors = FALSE)
checkpoint(raw, "1. raw data")
s1 <- raw; s1$price <- suppressWarnings(as.numeric(s1$price))   # 'abc' silently becomes NA
checkpoint(s1, "2. price -> numeric")
s2 <- s1[s1$price > 10, ]                                  # NA in the test creates an NA row
checkpoint(s2, "3. filter price > 10")
s2$revenue <- s2$price * s2$qty
checkpoint(s2, "4. add revenue")
cat("Total revenue:", sum(s2$revenue, na.rm = TRUE), "\n")
```

Result (verified): the NA cells grew from 1 to 2 (conversion), to 4 (filter) and to 6 (revenue), so the total revenue of 128 was calculated from complete rows only; a checkpoint log makes this visible immediately.

Q4. Compare indexing and logical subsetting techniques for extracting data and evaluate their performance trade-offs.

Answer:

Positional indexing selects rows or columns by number or name; logical subsetting selects rows by a condition. Their trade-offs:

Technique	Example	Speed and memory	Strengths	Weaknesses
Positional	df[1:1000,]	Fastest; no scan	Direct and simple	Breaks if rows or columns are reordered
By name	df[, "value"]	Fast	Robust to column order; readable	Column must exist
Logical	df[df$value > 2,]	Scans the column (O(n)) and creates a logical vector	Expressive condition-based selection	NA in the test gives NA rows
which()	df[which(df$value > 2),]	Similar to logical	Drops NA automatically	One extra step
subset()	subset(df, value > 2)	Slightly slower	Very readable; handles NA safely	Non-standard evaluation; avoid inside functions
data.table keys	dt[value > 2]	Very fast for repeated filters	Optimised search	Needs a package

Evaluation: positional indexing is the fastest but brittle; logical subsetting costs a scan of the column but is self-documenting and does not depend on row order, which makes it the safer default. Combining it with which() (or subset()) removes the NA problem, and data.table is the best choice when the same large table is filtered many times.

Example code (R):

```
set.seed(4)
n <- 1e6
df <- data.frame(id = 1:n, value = rnorm(n), grp = sample(letters[1:4], n, TRUE))
a <- system.time(r1 <- df[1:1000, ])["elapsed"]                # positional
b <- system.time(r2 <- df[df$value > 2, ])["elapsed"]          # logical
c2 <- system.time(r3 <- df[which(df$value > 2), ])["elapsed"]  # which()
d <- system.time(r4 <- subset(df, value > 2))["elapsed"]       # subset()
print(data.frame(method = c("df[1:1000, ]", "df[df$value > 2, ]", "df[which(...), ]", "subset()"),
                 seconds = c(a, b, c2, d), row.names = NULL))
cat("Rows returned by the three filters:", nrow(r2), nrow(r3), nrow(r4), "\n")
```

Result (verified): the three condition-based filters (logical, which() and subset()) returned exactly the same rows; the timing table printed by the code shows positional indexing is fastest and subset() slowest on a million rows.

Q5. Design a workflow to monitor and validate modifications made to a shared data frame by multiple users.

Answer:

Workflow designed so that several users can modify a shared data frame without losing control of the data:

1. Single source of truth. Master data kept in a shared database or protected file; the raw data are never edited, users work on copies.

2. Controlled changes through one function. All edits pass through apply_change(), which records the user, time, column and the rows affected.

3. Validation at every change. Unique key, allowed ranges, types and row count are checked with stopifnot(); invalid edits are rejected.

4. Versioned snapshots and checksums. saveRDS() creates a timestamped snapshot, and tools::md5sum() detects any unauthorised change to the file.

5. Audit log and difference reports. The log is saved with write.csv(); two versions are compared by a key-based merge or all.equal() to see exactly what changed.

6. Concurrency control. One editor at a time (lock file) or merge changes by key so that one user does not overwrite another.

7. Review and rollback. Peer approval before changes are published, scripts kept in Git, and an earlier snapshot restored with readRDS() if needed.

Example code (R):

```
shared <- data.frame(id = 1:5, status = c("open", "open", "closed", "open", "closed"),
                     amount = c(100, 200, 150, 300, 250), stringsAsFactors = FALSE)
audit_log <- data.frame(time = character(), user = character(), column = character(),
                        rows_changed = integer(), row_ids = character(), stringsAsFactors = FALSE)


apply_change <- function(df, log, user, column, rows, new_value) {
  before <- df
  df[rows, column] <- new_value
  stopifnot(!anyDuplicated(df$id), all(df$amount >= 0), nrow(df) == nrow(before))   # validation rules
  changed <- which(before[[column]] != df[[column]])
  log <- rbind(log, data.frame(time = format(Sys.time(), "%Y-%m-%d %H:%M:%S"), user = user,
                               column = column, rows_changed = length(changed),
                               row_ids = paste(df$id[changed], collapse = ","),
                               stringsAsFactors = FALSE))
  list(data = df, log = log)
}
r1 <- apply_change(shared, audit_log, "alice", "status", 2, "closed")
r2 <- apply_change(r1$data, r1$log, "bob", "amount", c(1, 4), c(120, 310))
print(r2$log)
snap <- tempfile(fileext = ".rds"); saveRDS(r2$data, snap)          # versioned snapshot
cat("Snapshot checksum:", unname(tools::md5sum(snap)), "\n")
res <- try(apply_change(r2$data, r2$log, "carol", "amount", 3, -50), silent = TRUE)
cat("Invalid edit by carol rejected:", inherits(res, "try-error"), "\n")
```

Result (verified): the audit log shows that alice changed row 2 of status and bob changed rows 1 and 4 of amount, a snapshot checksum was produced, and carol's invalid edit (a negative amount) was rejected by the validation rule.

Data Reshaping

Q1. Analyze a business dataset before and after reshaping. Evaluate how the transformation affects data interpretation.

Answer:

A business dataset that records sales per product for four quarters is normally stored in wide (report) form: one row per product and one column per quarter. After reshaping with melt(), it becomes long form with one row per product-quarter observation.

Aspect	Before (wide)	After (long)
Layout	3 rows x 5 columns (Product, Q1-Q4)	12 rows x 3 columns (Product, Quarter, Sales)
Quarter	Hidden in the column names	An explicit variable
Best for	Reading and reporting	Grouping, plotting, filtering and modelling
Summaries	Row-wise or column-wise sums	aggregate(Sales ~ Quarter) or by Product
Adding Q5	New column	New rows
Information	Total sales 2175	Total sales 2175 (no information lost)

Effect on interpretation: when all identifier variables are kept, no information is lost, but the unit of analysis changes from "product" to "product-quarter", so averages and row counts must be read accordingly (for example mean(Sales) is now the average per product per quarter). The totals before and after must be checked to prove the transformation is correct.

Example code (R):

```
library(reshape2)
sales_wide <- data.frame(Product = c("Laptop", "Phone", "Tablet"),
                         Q1 = c(120, 300, 80), Q2 = c(150, 280, 95),
                         Q3 = c(130, 320, 70), Q4 = c(170, 350, 110))
sales_long <- melt(sales_wide, id.vars = "Product", variable.name = "Quarter", value.name = "Sales")
print(sales_long)
cat("Total sales - wide:", sum(sales_wide[, -1]), "| long:", sum(sales_long$Sales), "\n")
cat("\nSales by quarter:\n");  print(aggregate(Sales ~ Quarter, sales_long, sum))
cat("\nAverage by product:\n"); print(aggregate(Sales ~ Product, sales_long, mean))
```

Result (verified): the total is 2175 in both formats; sales by quarter are 500, 525, 520 and 630 (Q4 is the highest); and Phone has the highest average sales per quarter (312.5).

Q2. A dataset must be reshaped for machine learning. Determine which structure is most suitable and justify your decision.

Answer:

Most machine-learning methods (lm, glm, randomForest, xgboost, k-nearest neighbours) expect one row per observation (customer, patient, machine) and one column per feature plus a target column. The most suitable structure is therefore the wide feature matrix with one row per entity.

Why. Rows must be independent observations with the same set of columns; wide-per-entity data fit this requirement directly, and categorical features can be one-hot encoded with model.matrix().

When raw data are long. If the source has many transactions per customer, they are aggregated and cast with dcast(customer ~ category, fun.aggregate = sum, fill = 0) so that every customer becomes one row.

When long format is still right. Sequence or time-series models and mixed-effects models for repeated measures use long data, with a subject identifier.

Extra steps. Handle NA, scale numeric features if the algorithm needs it, keep the target in its own column, and split train and test sets by entity so that the same customer is not in both.

Example code (R):

```
library(reshape2)
tx <- data.frame(customer = rep(c("C1", "C2", "C3"), times = c(3, 2, 3)),
                 category = c("Food", "Tech", "Food", "Tech", "Tech", "Food", "Food", "Home"),
                 amount   = c(200, 900, 150, 1100, 700, 300, 250, 400))
cat("Raw (long) transactions:", nrow(tx), "rows\n")
features <- dcast(tx, customer ~ category, value.var = "amount", fun.aggregate = sum, fill = 0)
features$churned <- c(0, 1, 0)                    # target label, one value per customer
print(features)
cat("ML-ready table: one row per customer =", nrow(features), "rows\n")
```

Result (verified): 8 transaction rows became 3 rows (one per customer) with the columns Food, Home, Tech and the churn label, which is the structure a model needs.

Q3. Evaluate the risks of reshaping data incorrectly in a healthcare analytics application.

Answer:

Incorrect reshaping of healthcare data can silently change clinical facts, so the risks are higher than in business data:

Wrong ID variables. Leaving visit out of id.vars turns visit numbers into measurements (see the output of the code).

Records attached to the wrong patient. Reshaping on a non-unique or mistyped patient key mixes the data of different patients.

Silent aggregation. dcast() without fun.aggregate defaults to length; using mean() can hide a dangerous single reading (for example one very high blood pressure).

Lost or invented values. na.rm in melt() drops missing readings, and fill = 0 in dcast() turns "not measured" into a real zero.

Duplicated or dropped rows. Counts of patients, events and prevalence rates become wrong.

Type mixing. melt() coerces all measure columns to one common type, so numeric values may become text.

Data leakage. Several rows of the same patient in both training and test sets exaggerate model accuracy.

Compliance and safety. Wrong clinical decisions, privacy risk and failure to meet data-accuracy requirements.

Safeguards: always keep patient_id and visit as identifier variables; check row counts (patients x measures); perform a round-trip test (melt, then dcast, must return the original); validate clinical ranges; keep an audit log; and have the result reviewed by a clinician.

Example code (R):

```
library(reshape2)
vitals <- data.frame(patient_id = c("P1", "P2"), visit = c(1, 1),
                     bp_sys = c(120, 150), heart_rate = c(72, 95))
m    <- melt(vitals, id.vars = c("patient_id", "visit"))
back <- dcast(m, patient_id + visit ~ variable, value.var = "value")
cat("Round trip (melt -> dcast) restores the data:",
   isTRUE(all.equal(vitals, back, check.attributes = FALSE)), "\n")
cat("Row count check (patients x measures):", nrow(m) == nrow(vitals) * 2, "\n")
wrong <- melt(vitals, id.vars = "patient_id")          # mistake: visit left out of id.vars
cat("\nWrong reshape - 'visit' is now treated as a measurement:\n")
print(wrong)
```

Result (verified): the round trip restored the original data (TRUE) and the row count check passed, whereas the incorrect melt shows visit as if it were a clinical measurement.

Q4. Compare wide and long data formats for time-series analysis and recommend the better approach for forecasting.

Answer:

For time-series analysis the two formats are compared below:

Aspect	Wide format	Long format
Layout	One column per series, one row per time point	One row per (time, series) pair

Suits	A few aligned series; ts() objects; multivariate models such as VAR	Many series, irregular timestamps, panel data
Missing time points	NA cells	Rows are simply absent
Adding a series	New column	New rows
Forecast every series	Loop over columns	Group by the series identifier and apply the model to each group
Plotting	matplot / ts.plot	ggplot colours and facets by series

Recommendation: the long format is the better working format for forecasting, because it stores any number of series, copes with missing and irregular timestamps, allows lag and rolling features to be created by group, and makes it easy to fit a model per series. The data are converted to the wide (ts matrix) form only for models that need aligned multivariate input, as shown in the code.

Example code (R):

```
dates <- seq(as.Date("2026-01-01"), by = "month", length.out = 6)
set.seed(8)
long <- data.frame(date = rep(dates, 2), series = rep(c("StoreA", "StoreB"), each = 6),
                   sales = round(c(100 + cumsum(rnorm(6, 2, 1)), 80 + cumsum(rnorm(6, 1, 1))), 1))
cat("LONG format (first rows):\n"); print(head(long, 4))
wide <- reshape(long, idvar = "date", timevar = "series", direction = "wide")
ts_mat <- ts(wide[, -1], start = c(2026, 1), frequency = 12)    # aligned multivariate series
cat("\nWIDE format as a multivariate time series:\n"); print(ts_mat)
cat("\nLast observation of every series, taken from the long format:\n")
print(sapply(split(long$sales, long$series), function(x) tail(x, 1)))
```

Result (verified): the same six monthly observations of two stores appear as 12 long rows and as a 6 x 2 multivariate ts object; the last observation of each series can be taken directly from the long table.

Q5. Design a reshaping strategy for integrating survey responses collected across multiple years.

Answer:

Survey files from different years usually differ in column names, number of questions and answer scales. The reshaping strategy is:

1. Question dictionary. Map old question names to standard names, recode scales (for example 1-5 and 1-10) and list the questions that exist only in some years.

2. Standardise each year. Rename columns in every yearly file and add a year column.

3. Stack the years. rbind() the common questions; keep year-specific questions (such as new_service) in a separate table.

4. Unique respondent key. Combine year and respondent number (paste(year, id)) unless the same panel respondents are followed over time.

5. Melt to long form. Columns resp_uid, year, question and score, so new questions or years only add rows.

6. Cast for reporting. dcast(question ~ year, fun.aggregate = mean) gives year-on-year comparison.

7. Validate and store. Check counts per year, ranges and missing values; read the yearly files with read.csv() and save the combined result with write.csv() or saveRDS().

Example code (R):

```
library(reshape2)
y2024 <- data.frame(ID = 1:3, Q1_satisfaction = c(4, 5, 3), Q2_recommend = c(5, 4, 3))
y2025 <- data.frame(ID = 1:3, satisfaction = c(5, 4, 4), recommend = c(4, 4, 5), new_service = c(3, 4, 4))
names(y2024) <- c("resp_id", "satisfaction", "recommend")       # harmonise column names
names(y2025)[1] <- "resp_id"
y2024$year <- 2024; y2025$year <- 2025
common <- c("resp_id", "year", "satisfaction", "recommend")
survey <- rbind(y2024[, common], y2025[, common])               # stack the years
survey$resp_uid <- paste(survey$year, survey$resp_id, sep = "-") # unique respondent key
long <- melt(survey, id.vars = c("resp_uid", "year"),
             measure.vars = c("satisfaction", "recommend"),
             variable.name = "question", value.name = "score")
cat("Long survey table (first rows):\n"); print(head(long, 4))
cat("\nAverage score by question and year:\n")
print(dcast(long, question ~ year, value.var = "score", fun.aggregate = mean))
```

Result (verified): the two years were combined into a long table and summarised as a question x year table of mean scores (satisfaction 4.00 in 2024 and 4.33 in 2025; recommend 4.00 and 4.33).

Melting and Casting of Data

Q1. Analyze a dataset where repeated measurements are stored in multiple columns. Determine how melting improves analysis.

Answer:

When repeated measurements are stored in several columns (Week1, Week2, Week3, Week4), the variable "week" is hidden in the column names. melt() stacks the columns so that every row is one measurement of one patient in one week. This improves analysis because:

One measurement per row (tidy). Every value has its patient and week beside it.

Simple grouped summaries. aggregate(BP ~ Week, ...) gives the mean per week in one call.

Easy plots. A single plotting call can draw a line per patient across weeks.

Per-patient change. The change from first to last week can be calculated by group.

Suitable for statistical models. Repeated-measures ANOVA and mixed models need long data.

Scales without code changes. If a Week5 column appears, the same code still works.

Example code (R):

```
library(reshape2)
bp <- data.frame(patient = c("P1", "P2", "P3"),
                 Week1 = c(150, 160, 140), Week2 = c(145, 158, 138),
                 Week3 = c(140, 150, 135), Week4 = c(135, 148, 130))
long <- melt(bp, id.vars = "patient", variable.name = "Week", value.name = "BP")
cat("Wide:", nrow(bp), "rows x", ncol(bp), "cols  ->  Long:", nrow(long), "rows x", ncol(long), "cols\n")
cat("\nMean BP per week:\n"); print(aggregate(BP ~ Week, long, mean))
cat("\nChange from Week1 to Week4 per patient:\n")
print(aggregate(BP ~ patient, long, function(x) x[length(x)] - x[1]))
```

Result (verified): the 3 x 5 wide table became 12 rows x 3 columns; the mean blood pressure fell from 150.0 in week 1 to 137.7 in week 4, and every patient improved by 10 to 15 units.

Q2. Evaluate the impact of casting data into different summary structures for business reporting.

Answer:

In dcast() the formula decides the layout (what forms the rows and the columns) and fun.aggregate decides what each cell means. Different casts of the same data therefore give different business messages:

Cast structure	Formula and function	Business use	Caution

Region x Product totals with margins	Region ~ Product, sum, margins = TRUE	Sales report with row and column totals	Shows size, not typical value
Month x Region averages	Month ~ Region, mean	Compare performance over time	Average ignores volume
Region x (Month, Product)	Region ~ Month + Product, sum, fill = 0	Detailed management pack	Very wide; fill = 0 treats missing as zero

Impact: sum shows total size, mean shows the typical value, and length (the default when no function is given) shows only how many records exist. The choice of rows and columns decides which comparison is easy; too many combinations create a wide, sparse table that is hard to read; and the fill value changes the interpretation of empty cells. A report should state the function used and how missing combinations are treated.

Example code (R):

```
library(reshape2)
set.seed(12)
sales <- data.frame(Region  = rep(c("North", "South"), each = 6),
                    Product = rep(c("A", "B", "C"), times = 4),
                    Month   = rep(rep(c("Jan", "Feb"), each = 3), times = 2),
                    Revenue = round(runif(12, 100, 500)))
cat("1) Region x Product, total revenue (with margins):\n")
print(dcast(sales, Region ~ Product, value.var = "Revenue", fun.aggregate = sum, margins = TRUE))
cat("\n2) Month x Region, average revenue:\n")
print(dcast(sales, Month ~ Region, value.var = "Revenue", fun.aggregate = mean))
cat("\n3) Region x (Month, Product), total revenue, empty cells = 0:\n")
print(dcast(sales, Region ~ Month + Product, value.var = "Revenue", fun.aggregate = sum, fill = 0))
```

The code produces the three summary structures from one data set; the first one includes the row and column totals ((all) row and column) that a manager needs for a quick overview.

Q3. Compare two casting approaches that produce different aggregation results and explain the reasons.

Answer:

Two casting approaches can give different numbers because of (a) the aggregation function, (b) the level at which the aggregation is done, and (c) the treatment of NA values. A deterministic example: Region A has four sales of 10 in store s1 and one sale of 50 in store s2.

Approach	Result	Reason

Mean of all five records (fun.aggregate = mean)	18	Every record has equal weight, so store s1 (4 records) dominates
Mean of the two store means (cast by Store, then average)	30	Every store has equal weight (average of averages)
Total sales (fun.aggregate = sum)	90	Different statistic: size instead of average
One NA present, mean without na.rm	NA	NA propagates through mean()
One NA present, mean with na.rm = TRUE	20	The NA is ignored (80 / 4)

The mean of group means equals the overall mean only when all groups have the same number of records. If no function is supplied, dcast() uses length and returns counts. Therefore the analyst must decide what each number should mean (record-weighted or group-weighted, total or average), choose the function and aggregation level accordingly, and document the na.rm setting.

Example code (R):

```
library(reshape2)
d <- data.frame(Region = "A", Store = c(rep("s1", 4), "s2"), Sales = c(10, 10, 10, 10, 50))
direct   <- dcast(d, Region ~ ., value.var = "Sales", fun.aggregate = mean)
by_store <- dcast(d, Region ~ Store, value.var = "Sales", fun.aggregate = mean)
cat("Approach 1 - mean of all 5 sales records     :", direct[1, 2], "\n")
cat("Approach 2 - mean of the two store means     :", rowMeans(by_store[, -1]), "\n")
cat("Approach 3 - total sales (fun.aggregate=sum)  :",
   dcast(d, Region ~ ., value.var = "Sales", fun.aggregate = sum)[1, 2], "\n")
d$Sales[2] <- NA
cat("\nWith one NA, mean without na.rm :", dcast(d, Region ~ ., value.var = "Sales", fun.aggregate =
mean)[1, 2], "\n")
cat("With one NA, mean with na.rm=TRUE:",
   dcast(d, Region ~ ., value.var = "Sales", fun.aggregate = mean, na.rm = TRUE)[1, 2], "\n")
```

Result (verified): 18 against 30 for the same data, 90 for the sum, and NA against 20 depending on na.rm - the differences are caused only by the aggregation choices.

Q4. Design a workflow using melting and casting to prepare data for predictive modeling.

Answer:

Workflow that uses melting and casting to prepare data for predictive modelling (predicting machine failure from sensor readings):

1. Import and fix types. read.csv() with colClasses; check NA and units.

2. Melt. melt(readings, id.vars = "machine") gives one reading per row for all sensors.

3. Engineer features by casting. dcast() with different fun.aggregate values (mean, max, sd) creates summary features for each sensor; the columns are renamed (temp_mean, vib_max ...) and merged by machine.

4. Add the target. Merge with the failure label (one value per machine).

5. Clean and prepare. Handle NA and outliers, encode categorical variables, scale numeric features if needed.

6. Split by entity. Divide training and test sets by machine, not by row, to avoid leakage.

7. Model and save. Fit glm() or randomForest, evaluate, and save the feature table with write.csv() or saveRDS().

Example code (R):

```
library(reshape2)
set.seed(21)
readings <- data.frame(machine = rep(paste0("M", 1:4), each = 5),
                       temp = round(rnorm(20, 70, 5), 1), vib = round(rnorm(20, 3, 0.6), 2))
long <- melt(readings, id.vars = "machine", variable.name = "sensor", value.name = "reading")
make_feature <- function(fun, tag) {
  f <- dcast(long, machine ~ sensor, value.var = "reading", fun.aggregate = fun)
  names(f)[-1] <- paste0(names(f)[-1], "_", tag); f }
features <- Reduce(function(a, b) merge(a, b, by = "machine"),
                   list(make_feature(mean, "mean"),
                        make_feature(function(x) if (length(x)) max(x) else NA_real_, "max"),
                        make_feature(sd, "sd")))
labels <- data.frame(machine = paste0("M", 1:4), failed = c(0, 1, 0, 1))
model_data <- merge(features, labels, by = "machine")
print(round(model_data[, -1], 2)); cat("\nOne row per machine:", nrow(model_data), "rows\n")
```

Result (verified): 20 raw readings were converted into a table with one row per machine, 6 engineered features (mean, max and standard deviation of temperature and vibration) and the failure label - the format required by predictive models.

Q5. Assess the challenges of applying melt and cast operations on large-scale industrial datasets.

Answer:

Applying melt and cast to large industrial data brings technical problems:

Challenge	Cause	Mitigation

Memory growth	melt() repeats the id columns for every measure column; 20,000 rows x 50 sensors become 1,000,000 rows	Select needed columns first, use integer or factor types, use data.table::melt
Speed	reshape2 works on a single thread and copies the data	Use data.table melt/dcast; process in chunks
Extremely wide casts	dcast() can create thousands of mostly-NA columns	Limit categories, keep data long, use sparse matrices (Matrix package)
Duplicate ID combinations	Need fun.aggregate; the default length silently returns counts	Check duplicates first and state the function explicitly
Mixed data types	melt() forces all measure columns into one type (often character)	Melt numeric and text columns separately
Missing or irregular readings	Sensor dropouts create NA and missing combinations	Use na.rm, fill and explicit rules
Data larger than RAM	The table cannot be loaded at once	Read in chunks (fread with nrows/skip), use a database or parquet files
Loss of time order	Casting sorts the rows	Keep the timestamp as an id and sort explicitly

Example code (R):

```
library(reshape2)
set.seed(30)
n_rows <- 20000; n_sensors <- 50
wide <- as.data.frame(matrix(rnorm(n_rows * n_sensors), nrow = n_rows))
names(wide) <- paste0("S", 1:n_sensors); wide$id <- 1:n_rows
t_melt <- unname(system.time(long <- melt(wide, id.vars = "id"))["elapsed"])
cat("Wide:", nrow(wide), "x", ncol(wide), "  ->  Long:", nrow(long), "x", ncol(long), "\n")
cat("Memory (MB)  wide:", round(as.numeric(object.size(wide)) / 1024^2, 1),
    " long:", round(as.numeric(object.size(long)) / 1024^2, 1), "\n")
cat("melt() took", t_melt, "seconds for", format(nrow(long), big.mark = ","), "rows\n")
```

Result (verified): the 20,000 x 51 wide table became a 1,000,000 x 3 long table and the memory size roughly doubled (7.7 MB to 15.3 MB), which shows how quickly the problem grows with real industrial data.

Merging Data Frames

Q1. Analyze the consequences of performing an incorrect join operation between two financial datasets.

Answer:

An incorrect join between financial datasets produces wrong numbers that look correct:

Duplicate keys multiply rows. A many-to-many or one-to-many join repeats balances, so totals are inflated (in the example the left join gives 5500 against the true 4500).

Inner join drops records. Accounts without transactions and transactions without accounts disappear, so totals are understated (inner join gives 4000).

Wrong or inconsistent key. Joining on names instead of IDs, different types, lost leading zeros, upper/lower case or spaces matches the wrong records or none.

NA treated as zero. Unmatched rows from left or outer joins contain NA, which can be mistaken for zero amounts.

Unintended join columns. merge() without by uses all common column names.

Consequences: misstated balances and revenue, incorrect risk or credit decisions, failed reconciliation, audit and regulatory problems, and undetected fraud because orphan transactions vanish. Prevention: check key uniqueness with duplicated(), standardise the keys, choose the join type on purpose, list unmatched records with an anti-join, compare row counts and control totals before and after the join, and add stopifnot() checks.

Example code (R):

```
accounts <- data.frame(acct = c("A1", "A2", "A3"), balance = c(1000, 2000, 1500))
txn      <- data.frame(acct = c("A1", "A1", "A2", "A4"), amount = c(50, 70, 30, 99))
cat("True total balance        :", sum(accounts$balance), "\n")
inner <- merge(accounts, txn, by = "acct")
cat("After inner join          :", sum(inner$balance), "(A1 counted twice, A3 dropped)\n")
left  <- merge(accounts, txn, by = "acct", all.x = TRUE)
cat("After left join           :", sum(left$balance), "(A1 counted twice)\n")
cat("\nTransactions with no matching account (anti-join):\n")
print(txn[!txn$acct %in% accounts$acct, ])
cat("Duplicate keys in txn$acct:", sum(duplicated(txn$acct)), "\n")
```

Result (verified): the true total balance is 4500, the inner join gives 4000 (A1 counted twice, A3 dropped), the left join gives 5500 (A1 counted twice), and the anti-join finds the orphan transaction for account A4.

Q2. Evaluate inner, outer, left, and right joins for integrating customer and transaction data.

Answer:

The four joins differ in which unmatched rows are kept (all.x and all.y arguments of merge()):

Join	R code	Rows kept	Use for customer and transaction data
Inner	merge(x, y, by)	Only customers that have transactions	Sales analysis of actual purchases; drops non-buyers and orphan transactions
Left	all.x = TRUE	All customers plus their transactions	Customer analysis: inactive customers (NA), churn, segmentation - recommended base table
Right	all.y = TRUE	All transactions even without a customer	Data-quality check for orphan transactions
Full outer	all = TRUE	Everything from both tables	Reconciliation and audit

Recommendation: use a left join from customers to transactions for customer analytics (after aggregating transactions per customer to avoid multiplying rows); use the inner join for pure sales totals; and use right or full outer joins as validation tools to detect orphan or missing records.

Example code (R):

```
customers <- data.frame(cust_id = 1:4, name = c("Arun", "Priya", "Kumar", "Divya"))
trans     <- data.frame(txn_id = 101:105, cust_id = c(1, 1, 2, 5, 5), amount = c(200, 150, 300, 80, 90))
joins <- list(inner = merge(customers, trans, by = "cust_id"),
              left  = merge(customers, trans, by = "cust_id", all.x = TRUE),
              right = merge(customers, trans, by = "cust_id", all.y = TRUE),
              full  = merge(customers, trans, by = "cust_id", all = TRUE))
cat("Rows returned by each join:\n"); print(sapply(joins, nrow))
cat("\nCustomers with no purchases (from the left join):\n")
print(joins$left[is.na(joins$left$txn_id), c("cust_id", "name")])
cat("\nOrphan transactions with no customer record (from the right join):\n")
print(joins$right[is.na(joins$right$name), c("txn_id", "cust_id", "amount")])
```

Result (verified): the joins return 3 (inner), 5 (left), 5 (right) and 7 (full) rows; the left join shows customers Kumar and Divya with no purchases, and the right join shows two orphan transactions for customer 5 who has no customer record.

Q3. A merged data frame contains unexpected duplicate rows. Investigate the possible causes and propose solutions.

Answer:

Unexpected duplicate rows after merge() are caused by:

Duplicate keys in one table. If a key appears twice in the lookup table, every matching row of the other table is repeated (C1 appears twice in the example).

Duplicates in both tables. A many-to-many match produces all combinations inside the key group.

A key that is not unique. Merging on a partial key (customer only instead of customer and date).

Unintended join columns. merge() without by uses every common column name.

Duplicates already in the source. Double loading or repeated extracts.

Solutions: detect duplicates before merging with anyDuplicated(); keep one row per key (!duplicated()) or aggregate the lookup table; use the complete composite key; always specify by explicitly; guard the result with stopifnot(nrow(result) == nrow(left_table)); and investigate the source of duplicates.

Example code (R):

```
orders <- data.frame(order_id = c(1, 2, 3), cust = c("C1", "C2", "C1"), amount = c(100, 200, 50))
cust   <- data.frame(cust = c("C1", "C1", "C2"), city = c("Chennai", "Delhi", "Mumbai"))  # C1 listed twice
cat("Orders:", nrow(orders), "| duplicate keys in lookup table:", sum(duplicated(cust$cust)), "\n")
m <- merge(orders, cust, by = "cust")
cat("Rows after merge:", nrow(m), "(expected", nrow(orders), ")\n"); print(m)
cust_clean <- cust[!duplicated(cust$cust), ]                # keep one record per key
m2 <- merge(orders, cust_clean, by = "cust")
stopifnot(nrow(m2) == nrow(orders))                         # guard: no row multiplication
cat("\nAfter de-duplicating the lookup table:", nrow(m2), "rows\n"); print(m2)
```

Result (verified): 3 orders became 5 rows after the merge because customer C1 was listed twice; after removing the duplicate lookup row the merge returned the expected 3 rows and the stopifnot() guard passed.

Q4. Design a strategy to merge data frames from multiple departments while ensuring data consistency.

Answer:

Strategy for merging data frames from several departments while keeping the data consistent:

1. Agree the master key and data ownership. For example employee ID with a common definition.

2. Profile each table. Check columns, types, duplicates and NA.

3. Standardise. Column names (tolower), key format (trimws, toupper), types, units and category codes.

4. Resolve conflicting columns. Rename with department prefixes or suffixes and define which department wins when values differ.

5. Merge step by step. Reduce(function(a, b) merge(a, b, by = "emp_id", all = TRUE), list(...)), starting with the most reliable table.

6. Validate each step. Unique keys, expected row counts and a list of unmatched keys.

7. Reconcile and document. Compare totals (headcount, salary) with the source systems, keep lineage notes, and save with write.csv() or saveRDS().

Example code (R):

```
clean_key <- function(x) toupper(trimws(x))                    # standardise the key
hr  <- data.frame(emp_id = c("E1", "E2", "E3"), name = c("Arun", "Priya", "Kumar"), dept = c("HR", "IT", "IT"))
fin <- data.frame(EMP_ID = c(" e1", "E2 ", "E4"), salary = c(50000, 62000, 45000))
ops <- data.frame(emp_id = c("E1", "E3", "E4"), projects = c(2, 4, 1))
names(fin) <- tolower(names(fin))                              # standardise column names
hr$emp_id <- clean_key(hr$emp_id); fin$emp_id <- clean_key(fin$emp_id); ops$emp_id <- clean_key(ops$emp_id)
for (d in list(hr, fin, ops)) stopifnot(!anyDuplicated(d$emp_id))   # key must be unique per department
merged <- Reduce(function(a, b) merge(a, b, by = "emp_id", all = TRUE), list(hr, fin, ops))
stopifnot(!anyDuplicated(merged$emp_id))
print(merged)
cat("\nEmployees missing from at least one department:", sum(!complete.cases(merged)), "\n")
```

Result (verified): the HR, Finance and Operations tables were merged into one table of four employees (E1 to E4) after cleaning the keys; 3 employees are missing from at least one department (E4 has no HR record, E2 has no project data, E3 has no salary) and are flagged for follow-up.

Q5. Compare key-based merging and index-based merging techniques for large engineering datasets.

Answer:

Key-based merging joins rows by the values of key columns; index-based merging joins them by position (cbind) or by row names (merge with by = 0).

Aspect	Key-based merge	Index-based merge
Matching	By key values (merge(), joins, data.table keyed join)	By row position or row names
Row order	Irrelevant	Must be identical in both tables
Missing or extra rows	Detected as unmatched keys	Silently misaligned

Speed	Needs a lookup (data.table keys use fast binary search)	Fastest (no lookup)
Safety	High	Low: one sorted table gives wrong results
Typical use	Sensor logs by device ID and timestamp; test results by part number	Columns computed from the same data frame with guaranteed order

Recommendation: for large engineering datasets use key-based merging, preferably with data.table (setkey or on =) for speed, after checking that the keys are unique. Use index-based merging only when the row order is guaranteed and verified, for example with identical(ids_a, ids_b).

Example code (R):

```
sensors  <- data.frame(id = 1:5, temp = c(20.1, 21.4, 19.8, 22.0, 20.7))
meta     <- data.frame(id = 1:5, site = c("S1", "S2", "S3", "S4", "S5"))
meta_shuf <- meta[c(3, 1, 5, 2, 4), ]                    # same data, different row order
by_index <- cbind(sensors, site = meta_shuf$site)        # index (position) based
by_key   <- merge(sensors, meta_shuf, by = "id")         # key based
cat("Index-based result (WRONG - sites are misaligned):\n"); print(by_index)
cat("\nKey-based result (correct):\n"); print(by_key)
cat("\nRow order identical in both tables? ", identical(sensors$id, meta_shuf$id), "\n")
by_rowname <- merge(sensors, meta, by = 0)               # merge on row names (index) is safe only here
cat("Row-name merge works only because both tables are in the same order:", nrow(by_rowname), "rows\n")
```

Result (verified): when the second table was in a different order, cbind() attached the wrong sites to the sensors (id 1 received S3), while merge() by id returned the correct site for every sensor.

Appendix: File Input/Output Operations in R

Reading data from files and writing results back to files is part of every data frame workflow. The functions used in this report, and the common options that prevent type and encoding problems, are summarised below.

Function	Purpose	Example	Notes
read.csv() / write.csv()	Import / export comma-separated files	df <- read.csv("data.csv", stringsAsFactors = FALSE, na.strings = c("NA", "", "N/A")) write.csv(df, "clean.csv", row.names = FALSE)	Use colClasses to fix column types; row.names = FALSE avoids an extra index column
read.table() / write.table()	Files with another separator	read.table("log.txt", header = TRUE, sep = "|")	sep, header and quote control the format; read.delim() is for tab files
readLines() / writeLines()	Raw text lines (logs)	lines <- readLines("sensor.log")	Useful before parsing irregular files
saveRDS() / readRDS()	Store one R object in binary form	saveRDS(df, "df.rds"); df <- readRDS("df.rds")	Fast; keeps column types exactly
save() / load()	Several R objects in an .RData file	save(df1, df2, file = "work.RData")	load() restores the objects with their names
readxl::read_excel()	Import Excel sheets	read_excel("sales.xlsx", sheet = 1)	Package readxl
jsonlite::fromJSON()	Import JSON or API results	fromJSON("data.json")	Package jsonlite; returns lists or data frames
data.table::fread() / fwrite()	Very fast CSV import / export	dt <- fread("big.csv")	Suitable for large files; detects types automatically
DBI::dbConnect() / dbGetQuery()	Read data from a database	dbGetQuery(con, "SELECT * FROM sales")	Avoids loading unneeded data
read.csv(text = ...)	Treat a text string as a file	read.csv(text = "ID,Name\n1,Arun")	Used in the examples for self-contained code

File handling	Check and manage files	file.exists(), list.files(), getwd(), setwd(), tempfile(), file.remove()	Use relative paths and check file.exists() before reading

Good practice: keep the raw files unchanged and write cleaned data to a new file; declare missing-value codes (na.strings) and column types (colClasses) when importing; use a binary format (.rds) for intermediate results; and record the file checksum (tools::md5sum()) when the exact version of the data matters.